



دانشکده فنی و مهندسی

گزارش پروژه دوره کارشناسی

تشخیص عوارض جاده‌ها

دانشجویان:

مهدی براتی 99123588008

سید علی امامی 9912358005

استاد راهنما:

دکتر حسن ختن‌لو

بهمن 1403

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده

کیفیت نامناسب جاده‌ها و وجود چاله‌ها یکی از عوامل اصلی آسیب به وسایل نقلیه، کاهش ایمنی رانندگی و افزایش هزینه‌های نگهداری خودروها و زیرساخت‌های شهری است. در این راستا، پروژه با هدف تشخیص هوشمند ناهمواری‌های سطح جاده و ارائه راهکاری نوین برای بهبود مدیریت نگهداری راه‌ها توسعه یافته است.

این سیستم با بهره‌گیری از حسگرهای سنجش فاصله، ارتعاش، صوت، دوربین و GPS، داده‌های مرتبط با کیفیت جاده را جمع‌آوری کرده و با استفاده از الگوریتم‌های پردازش داده، بینایی ماشین و یادگیری ماشین، نقاط دارای چاله را شناسایی و تحلیل می‌کند. سپس، این اطلاعات در یک سامانه نرم‌افزاری تحت وب یا موبایل نمایش داده می‌شود تا سازمان‌های راه‌سازی، شهرداری‌ها و حتی رانندگان بتوانند از آن برای انتخاب مسیرهای ایمن‌تر و برنامه‌ریزی تعمیرات جاده‌ای استفاده کنند.

این پروژه در سه بخش اصلی پیاده‌سازی شده است:

1. تحلیل و انتخاب حسگرها: بررسی انواع حسگرهای مناسب برای تشخیص چاله‌ها و نحوه نصب و بهینه‌سازی عملکرد آن‌ها.
2. پردازش و تحلیل داده‌ها با پایتون: استفاده از الگوریتم‌های هوشمند برای تشخیص و فیلتر کردن ناهنجاری‌های جاده‌ای.
3. توسعه سامانه نرم‌افزاری: نمایش داده‌های پردازش‌شده در قالب نقشه‌های دیجیتال، امکان ثبت گزارش توسط کاربران و ارائه تحلیل‌های آماری برای مدیریت بهتر تعمیرات.

نتایج این پژوهش نشان می‌دهد که با استفاده از این سامانه می‌توان ایمنی جاده‌ها را افزایش داد، هزینه‌های تعمیر خودروها را کاهش داد و نگهداری زیرساخت‌های حمل‌ونقل را بهینه‌سازی کرد. این سیستم همچنین قابلیت توسعه برای شناسایی سایر نواقص جاده‌ای و مشکلات زیرساختی را داراست و می‌تواند به‌عنوان یک ابزار کارآمد در مدیریت هوشمند شهری مورد استفاده قرار گیرد.

فهرست مطالب

عنوان صفحه

4..... فصل یک) راهکار ها و حسگر ها

5..... 1-1 حسگر سنجش فاصله

9..... 2-1 حسگر ضربه و ارتعاش

13..... 3-1 حسگر تشخیص صوت

17..... 4-1 دوربین تشخیص دهنده عوارض

18..... 5-1 حسگر موقعیت مکانی

22..... فصل دو) پیاده سازی و نتایج

23..... 1-2 پردازش داده‌های حسگر سنجش فاصله

33..... 2-2 پردازش داده‌های حسگر ضربه و ارتعاش

41..... 3-2 پردازش داده‌های حسگر تشخیص صوت

49..... 4-2 پردازش داده‌های تصویری

56..... 5-2 حسگر موقعیت مکانی

59..... 6-2 امتیاز دهی و ترکیب داده ها

64..... فصل سه) چارچوب نرم افزاری

پیشگفتار

تعریف پروژه

با افزایش تعداد وسایل نقلیه و تردد جاده‌ای، کیفیت سطح جاده‌ها اهمیت دوچندانی پیدا کرده است. ناهمواری‌های سطحی، چاله‌ها و ترک‌های جاده‌ای می‌توانند موجب آسیب به خودروها، کاهش ایمنی رانندگی و حتی تصادفات شوند. این پروژه با هدف تشخیص هوشمند این نواقص طراحی شده است.

این سیستم با بهره‌گیری از پردازش داده‌های سنسوری و تصویری، قادر است اطلاعات دقیقی از وضعیت جاده‌ها جمع‌آوری کرده و چاله‌ها را شناسایی کند. داده‌های دریافت‌شده توسط حسگرهای مختلف و دوربین‌ها پردازش شده و اطلاعات موقعیت مکانی آن‌ها ثبت می‌گردد تا در اختیار سازمان‌های مربوطه و سایر ذی‌نفعان قرار گیرد.

اهداف پروژه

1. ارتقای ایمنی رانندگان و کاهش تصادفات
2. بهینه‌سازی تعمیر و نگهداری جاده‌ها
3. کاهش هزینه‌های نگهداری خودرو
4. بهبود مدیریت شهری و زیرساخت‌های حمل‌ونقل
5. ایجاد سامانه گزارش‌دهی و هشدار هوشمند

روش کار و نحوه پیاده سازی

برای تشخیص چاله ها، این سیستم از مجموعه ای از حسگرها و روش های پردازش تصویر استفاده می کند:

1. حسگرهای فیزیکی:

- حسگر سنجش فاصله (اولتراسونیک): برای اندازه گیری تغییرات سطحی جاده.
- حسگر ارتعاش و شتاب سنج: برای تشخیص ضربات ناگهانی هنگام عبور خودرو از روی چاله ها.
- حسگر صوتی: برای شناسایی تغییرات صوتی ایجاد شده در اثر عبور خودرو از چاله ها.

2. پردازش تصویری:

- استفاده از دوربین نصب شده روی خودرو برای تحلیل سطح جاده و شناسایی چاله ها از طریق پردازش تصویر.
- تشخیص تغییرات ناگهانی در بافت و رنگ سطح جاده با استفاده از الگوریتم های بینایی ماشین.

3. سیستم موقعیت یابی: (GPS)

- ثبت مختصات دقیق چاله ها و عوارض جاده ای برای نمایش روی نقشه های دیجیتال.
- امکان استفاده از این داده ها برای به روز رسانی نقشه های ناوبری و کمک به تعمیر جاده ها.

4. پردازش و تحلیل داده ها:

- ترکیب اطلاعات دریافتی از سنسورها و تصاویر برای تشخیص دقیق تر چاله ها.
- اعمال فیلترهای نرم افزاری برای حذف داده های نویزی و افزایش دقت شناسایی.

کاربردهای پروژه

این سیستم می‌تواند در بخش‌های مختلفی مورد استفاده قرار گیرد:

- شهرداری‌ها و سازمان‌های راه‌سازی:
 - برنامه‌ریزی و اجرای تعمیرات جاده‌ای بر اساس اطلاعات ثبت‌شده.
 - کاهش هزینه‌های نگهداری و افزایش کیفیت راه‌ها.
- شرکت‌های حمل‌ونقل عمومی و تاکسیرانی:
 - انتخاب مسیرهای کم‌خطرتر برای وسایل نقلیه عمومی.
 - کاهش هزینه‌های تعمیر و نگهداری خودروهای ناوگان حمل‌ونقل.
- شرکت‌های بیمه خودرو:
 - تحلیل میزان آسیب‌های ناشی از وضعیت جاده‌ای در ارزیابی خسارات بیمه‌ای.
- برنامه‌های مسیریابی و ناوبری:
 - نمایش مسیرهای بهتر و هشدار درباره مسیرهای دارای چاله برای رانندگان.

فصل اول

راهکار ها و حسگر ها

1-1 حسگر سنجش فاصله

یکی از ویژگی‌های اصلی عوارض جاده‌ای تغییر ناگهانی ارتفاع سطح جاده می‌باشد. این ویژگی موجود در مسئله می‌تواند ما را در یافتن فرورفتگی‌ها و ناهمواری‌ها یاری رساند. چالش‌هایی که در مورد این حسگر وجود دارد شامل **محل قرارگیری حسگر و دقت حسگر** می‌باشد.

محل قرارگیری حسگر

می‌توان این حسگر را در جلوی خودرو نصب نمود تا فاصله از زمین را بسنجد اما در سطح خیابان آب راه‌هایی وجود دارد که عمق زیادی دارند ولی عرض کمی دارند که ضربه و ناهمواری محسوب نمی‌شوند ولی چنانچه حسگر را در این نقطه وصل کنیم، داده‌هایی که از محیط می‌گیرد و تولید می‌کند که نویز دارند و جداسازی این نوع داده‌ها چالش بزرگی است زیرا ویژگی‌های مشابهی با چاله‌های خیابان دارد.

می‌توان این حسگر را در محل جلوبندی خودرو، قسمت فنر قرار داد تا چنانچه تغییر ارتفاعی در طول فنر وجود داشت آن را ذخیره کند. این کار باعث می‌شود تا چنانچه خودرو از روی پل یا آبراه‌های کوچک عبور کرد حسگر داده اشتباه ذخیره نکند و تنها زمانی حسگر داده ذخیره می‌کند که خودرو درون چاله یا ناهمواری بیفتد.

دقت حسگر

با در نظر گرفتن حداقل زمانی که چرخ خودرو درون یک چاله می افتد می توانیم به دست آوریم که حسگر با چه میزان بسامد اندازه گیری مناسب است.

ابعاد چرخ خودروهای شهری معمولاً بین ۱۳ تا ۱۸ اینچ می باشد و عرض لاستیک ها نیز عموماً بین اعداد ۱۶ تا ۲۵ سانتی متر می باشد. با توجه به ضریب فشردگی لاستیک در حین برخورد با چاله، تغییر طول فنر به میزان یک سانتی متر و یا بیشتر برای تشخیص یک چاله معیار مناسبی است (وزنی که روی هر چرخ در خودروهای شهری وجود دارد به طور معمول ۳۰۰ کیلوگرم است).

با توجه به فرضیات بالا و طبق محاسباتی که انجام شده است، طول چاله عددی بین ۱۰ الی ۱۵ سانتی متر باید باشد.

در این قسمت مشخص شد با نصب حسگر در محل مناسب می توانیم چاله هایی که تاثیر ناچیزی بر روی حمل و نقل دارند را در نظر نگیریم و در داده هایی که جمع آوری کردیم بار محاسباتی و پردازش سیستم را کاهش می دهیم.

با فرض حداکثر سرعت مجاز ۱۲۵ کیلومتر بر ساعت (34.7m/s) و طول چاله حداقل ۱۰ سانتی متر $0.0028s = 1 / 34.7$

0.0028s مدت زمانی است که خودرو طول می کشد تا چاله را رد کند. و تغییر طول حداقل یک سانتی متر فنر در صورت رخ می دهد که خودرو در وسط چاله باشد. لذا فقط 0.00028s تغییر طول فنر یک سانتی متر می رسد و حسگر باید با بسامدی بیش از 3600 داشته باشد تا بتواند چاله را ثبت کند.



تصویر 1-1

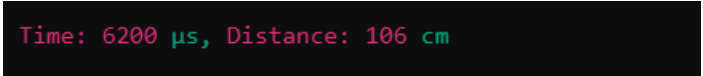
کاربرد ماژول فاصله سنج:

1. سیستم های امنیتی
2. سیستم های پارکینگ
3. ساخت ربات
4. مسیریابی
5. پروژه های اینترنت اشیا

مشخصات ماژول آلتراسونیک فاصله سنج IOE-SR05 (تصویر 1-1):

1. ولتاژ کاری DC 3.0-5.5V :
2. جریان کاری: 8 میلی آمپر
3. بسامد کاری: 40 کیلوهرتز
4. محدوده : 0 تا 2000 میلی متر
5. رزولوشن: 1 میلی متر
6. نرخ baud پورت سریال: 8,19600,n
7. مدت زمان پاسخگویی: 18 میلی ثانیه
8. اندازه های استاندارد: 33 * 22 * 15 میلی متر

قیمت مناسب و بسامد ۴۰ کیلو هرتزی این ماژول نیاز ما را مرتفع می کند.



Time: 6200 μ s, Distance: 106 cm

تصویر 2-1



Time: 9800 μ s, Distance: 168 cm

تصویر 3-1

1-2 حسگر ضربه و ارتعاش

حسگر ضربه و ارتعاش ابزاری است که برای شناسایی و اندازه‌گیری تکان‌ها، لرزش‌ها و ضربات مکانیکی استفاده می‌شود. این حسگرها می‌توانند تغییرات فیزیکی را به سیگنال‌های الکتریکی تبدیل کنند که برای پایش سلامت تجهیزات، تشخیص تصادفات، یا امنیت ساختمان‌ها به کار می‌رود.

محل قرارگیری حسگر ضربه

برای محل قرارگیری این حسگر به شدت ضربه و همچنین پایش کلی تمام ضربه‌ها توجه می‌کنیم. که یک حسگر روی تعلیق خودرو و یک حسگر در داخل کابین خودرو نصب می‌کنیم. حسگری که بر روی تعلیق خودرو نصب شده است تمام ضربه‌ها و تعاشات را ثبت می‌کند ولی حسگر داخل کابین تنها ضربه‌های شدید را که تعلیق خودرو توانایی جذب آن را نداشت احساس می‌کند. داده‌های به دست آمده از حسگر ضربه با در نظر گرفتن ویژگی زمان از پیوستگی خاصی برخوردار است که چنانچه تغییر ناگهانی ایجاد شود همانند تصویر 1-4 می‌توان آن را به عنوان ضربه در نظر گرفت.

Time (ms)	Acceleration (X, Y, Z) (m/s ²)
0	(0.01, 0.02, 9.81)
10	(0.03, 0.01, 9.79)
20	(5.12, 2.43, 8.95) <-- ضربه
30	(0.02, 0.01, 9.80)

تصویر 1-4

محورهای X, Y, Z در داده‌های حسگر ضربه و ارتعاش به سه بعد اصلی حرکت اشاره دارند:

X حرکت در جهت چپ و راست (محور افقی)

Y حرکت در جهت جلو و عقب (محور عمودی)

Z حرکت در جهت بالا و پایین (محور عمودی نسبت به سطح زمین)

این محورها معمولاً در شتاب‌سنج‌ها (Accelerometers) و ژيروسکوپ‌ها برای اندازه‌گیری ارتعاش، ضربه، تکان‌ها و حرکات استفاده می‌شوند. مقادیر آن‌ها معمولاً برحسب m/s^2 (متر بر مجذور ثانیه) گزارش می‌شود و مقدار Z معمولاً نزدیک به $9.81 m/s^2$ (جاذبه زمین) است.

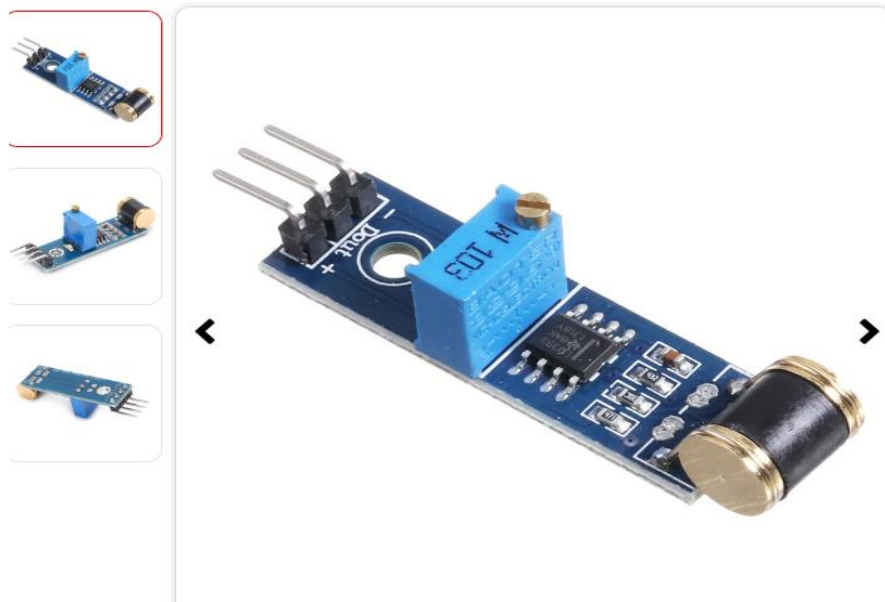
هنگامی که در سطح جاده ارتعاش ایجاد می‌شود، حرکت و شتاب به صورت عمودی می‌باشد لذا پارامتر Z را باید مورد بررسی قرار دهیم و مقدار X و Y برای ما معیار نیست.

انتخاب حسگر

با توجه به نوع مسئله و حسگرهای موجود در بازار ما باید از حسگری استفاده کنیم که خروجی به صورت آنالوگ باشد تا با توجه به شدت بتوانیم دسته‌بندی و پیش پردازش انجام دهیم تا محدوده خاصی از لرزه‌ها را تایید کنیم و بقیه را در نظر نگیریم.

Time (ms)	Vibration Level (0-1023)
0	0
100	5
200	500 <-- لرزش متوسط
300	1023 <-- لرزش شدید
400	0
500	700 <-- لرزش قوی
600	0

تصویر 1-5



تصویر 6-1

تصویر 6-1 ماژول حسگر لرزش زاویه S801 است که دارای دو پایه خروجی آنالوگ و دیجیتال مجزا است که هر یک از آن‌ها بسته به شدت لرزش دریافتی، یک خروجی ولتاژ (0 تا 5 ولت) در پایه A0 و یک سطح منطقی (0 یا 1) در پایه D0 در اختیار کاربر قرار می‌دهند.

نمونه داده تولیدی

Time (ms)	Analogue Output (0-1023)	Digital Output (0-1)	Notes
0	10	0	بدون لرزش (سطح پایه)
100	15	0	نویز جزئی
200	450	0	لرزش متوسط
300	1023	1	لرزش شدید یا تغییر زاویه زیاد
400	50	0	بازگشت به حالت عادی
500	600	0	تکان ناگهانی
600	20	0	بدون لرزش

تصویر 7-1

کاربرد های خروجی آنالوگ حسگر S801

اندازه گیری شدت لرزش یا تغییر زاویه
تنظیم حساسیت حسگر بر اساس مقادیر متغیر
استفاده در سیستم های هشدار دقیق تر
ترکیب با فیلتر نرم افزاری برای کاهش نویز

بسامد پاسخ دهی حسگر S801

بسامد کاری معمولی: تقریباً 1Hz تا 3kHz
زمان پاسخگویی: در حد چند میلی ثانیه

با توجه به بسامد اندازه گیری و نوع خروجی می توانیم از این حسگر استفاده کنیم.

1-3 حسگر تشخیص صوت

همانطور که می‌دانیم خودرو در حین حرکت صدا هایی تولید می‌کند که یک سری از صداها ناشی از برخورد باد با بدنه خودرو و همچنین بخشی از صداها ناشی از برخورد لاستیک‌ها با سطح جاده می‌باشد.

زمانی که لاستیک خودرو درون چاله می‌افتد صدای ناهنجاری تولید می‌کند که این صدا بسامد متفاوتی دارد. ما می‌توانیم با استفاده از حسگر تشخیص صوت و این تغییر بسامد برای پیدا کردن چاله کمک بگیریم.

چالش

در استفاده از حسگر صوتی به دلیل اینکه منابع متنوعی وجود دارد که می‌تواند صدا تولید کند داده‌ها دارای نویزها و خطاهای زیادی هستند لذا از داده‌های به دست آمده در این حسگر تنها به عنوان یک پارامتر فرعی استفاده می‌کنیم.



تصویر 8-1

تصویر 8-1 ماژول حسگر صدا KY-037 است که برای تشخیص صدا یا سیگنال‌های صوتی در محیط طراحی شده است. این ماژول از میکروفن الکترت کندانسور استفاده می‌کند تا تغییرات صوتی را ضبط کرده و آن‌ها را به یک سیگنال ولتاژ آنالوگ تبدیل کند که می‌تواند توسط میکروکنترلرها یا مدارهای دیجیتال دیگر پردازش شود.

این ماژول معمولاً در برنامه‌های مختلفی که در آن‌ها نیاز به تشخیص صدا وجود دارد مانند دستگاه‌های فعال‌شونده با صدا، سیستم‌های روشنایی فعال‌شونده با صدا و پروژه‌های پاسخگو به صدا استفاده می‌شود. این ماژول به دلیل سادگی و سهولت در ادغام با پروژه‌های الکترونیکی مختلف شناخته شده است البته این میکروفن برای ضبط صدا توصیه نمی‌شود.

ماژول دارای دو LED قرمز است. با اتصال ولتاژ تغذیه به ماژول LED پاور روشن میشود. با افزایش صدای محیط و رسیدن آن به آستانه تنظیم شده، LED دیگر روشن می شود. با استفاده از پتانسیومتر تعبیه شده روی ماژول میتوانید حساسیت حسگر را تنظیم نمایید.

از دیگر ویژگی های این ماژول این است که علاوه بر خروجی دیجیتال ، دارای یک خروجی آنالوگ نیز می باشد که به شما برای اندازه گیری شدت صوت نیز کمک می نماید.

مشخصات فنی ماژول حسگر صوت - حسگر صدا KY-037

- مدل: KY-037
- ولتاژ ورودی: $V \sim 5V3.3$
- خروجی ها: دارای دو خروجی AO (آنالوگ) و DO (دیجیتال)
- حساسیت : میکروفن با حساسیت بالا
- چراغ نشانگر: دارای نشانگر روشنایی
- سوراخ نصب: سوراخ 3 میلی متری برای نصب
- تعداد پین: 4 پین
- اندازه: 35mm x 15mm x 14mm

ویژگی‌های ماژول حسگر صوت - حسگر صدا KY-037

- تشخیص صدا: قادر به تشخیص صدا یا نویز در محیط.
- حساسیت قابل تنظیم: شامل پتانسیومتر برای سفارشی‌سازی حساسیت.
- خروجی آنالوگ: ارائه سیگنال ولتاژ آنالوگ متناسب با سطح صدای تشخیص داده شده است.

نمونه خروجی

```
Analog: 120 سکوت اولیه --<
Analog: 115
Analog: 110
Analog: 800 شروع داد زدن --<
Analog: 850
Analog: 870
Analog: 820
Analog: 300 صدا کاهش پیدا می‌کند --<
Analog: 150
Analog: 120 بازگشت به سکوت --<
Analog: 110
```

تصویر 9-1

با توجه به تصویر 9-1 زمانی که یک صدا با شدت بالا در محیط وجود دارد حسگر عدد بالاتری را نمایش می‌دهد و چنانچه عامل صدا قطع شود مقدار نمایشی کاهش می‌یابد.

1-4 دوربین تشخیص دهنده عوارض

استفاده از دوربین برای تشخیص عوارض در اماکن شهری و زمان‌هایی که خودرو از سرعت بسیار بالایی برخوردار نیست امکان‌پذیر است.

با توجه به دوربین‌های موجود در بازار بسامد آنها بین ۳۰ تا ۶۰ فریم بر ثانیه می‌باشد.

اگر بازه تصویربرداری یک متر باشد و دوربین 30 فریم بر ثانیه باشد. تصویربرداری از تمام نقاط جاده حداکثر تا سرعت ۳۰ متر بر ثانیه (108 km/h) به درستی انجام می‌شود.

با توجه به عوامل تاثیرگذار بر روی کیفیت تصاویر از جمله تار شدن، مات شدن و نور ناکافی در زمان شب از این پارامتر نیز به عنوان یک پارامتر فرعی کمک می‌گیریم.

5-1 حسگر موقعیت مکانی

حسگر موقعیت مکانی (GPS) به ما کمک می‌کند تا چاله و عوارض پیدا شده را با درج موقعیت مکانی ذخیره کنیم و بتوان برای ایجاد گزارش از آن استفاده کنیم.



نصیر 10-1

تصویر 10-1 ماژول GPS GY-NEO-6M ورژن 2 به همراه آنتن است که ماژول-GPS GY-NEO-6M V2 یک گیرنده GPS با دقت بالا و مصرف کم انرژی است که از ماژول NEO-6M شرکت u-blox بهره می‌برد. این ماژول برای کاربردهایی مانند ردیابی خودرو، پروژه‌های رباتیک، پهپادها و سیستم‌های ناوبری بسیار مناسب است.

ویژگی‌های ماژول GY-NEO-6M V2 :

تعداد کانال : ۵۰ کانال

دقت موقعیت‌یابی : کمتر از ۲.۵ متر در شرایط ایده‌آل

بسامد کاری : GPS: ۱.۵۷۵۴۲ گیگاهرتز (L1 Band)

فرمت خروجی دیتا: NMEA (GGA, GSV, GSA, RMC, VTG) UBX

سرعت انتقال داده (Baud Rate) : پیش‌فرض 9600 bps (قابل تغییر)

آنتن : دارای آنتن سرامیکی داخلی به علاوه آنتن اکسترنال با کانکتور u.FL

ولتاژ کاری : ۳.۳ ولت تا ۵ ولت

ارتباط سریال : UART (TX, RX)

مصرف انرژی : حدود 50mA

خروجی ماژول:

فرمت NMEA GPGGA رایج‌ترین استاندارد داده‌های GPS است و شامل چندین جمله (Sentence) می‌شود که هر کدام اطلاعات خاصی ارائه می‌دهند.

تصویر 11-1 نمایشی از این فرمت است که در تصویر 12-1 توضیحات جملات است.

```
$GPGGA,123519.00,3749.1832,N,12225.9530,W,1,08,0.9,545.4,M,46.9,M,,*47
```

تصویر 11-1

توضیح	مقدار	شماره
زمان 12:35:19 UTC - (hhmmss.ss)	123519.00	1
عرض جغرافیایی (°37 49.1832')	3749.1832	2
جهت عرض جغرافیایی (شمالی)	N	3
طول جغرافیایی (°122 25.9530')	12225.9530	4
جهت طول جغرافیایی (غربی)	W	5
کیفیت موقعیت (1 = GPS معمولی، 2 = DGPS، 0 = نامعتبر)	1	6
تعداد ماهواره‌های متصل	08	7
دقت افقی (HDOP)	0.9	8
ارتفاع از سطح دریا (متر)	545.4	9
واحد متر	M	10
ارتفاع ژئوید نسبت به سطح دریا	46.9	11
واحد متر	M	12
مقدار خالی (DGPS data)		13
Checksum (برای بررسی صحت داده‌ها)	47*	14

تصویر 12-1

نکته پایانی:

با توجه به اینکه ویژگی زمان در سایر حسگرها به صورت نسبی است و در حسگر سنجش موقعیت مکانی، زمان به صورت مطلق است می‌توانیم زمان روشن شدن خودرو را زمان شروع به کار کردن حسگرها در نظر بگیریم تا مقدار داده زمانی در GPS به صورت نسبی تبدیل شود تا با سایر داده‌ها هماهنگی داشته باشد.

در آخر با تشخیص عوارض و چاله‌های موجود در مسیر با استفاده از ویژگی زمان که در تمام حسگرها وجود دارد می‌توانیم موقعیت مکانی این عوارض را به دست آوریم و آن‌ها را بر روی نقشه نمایش دهیم و یا در یک لیست ذکر کنیم.

فصل دوم

پیاده سازی و نتایج

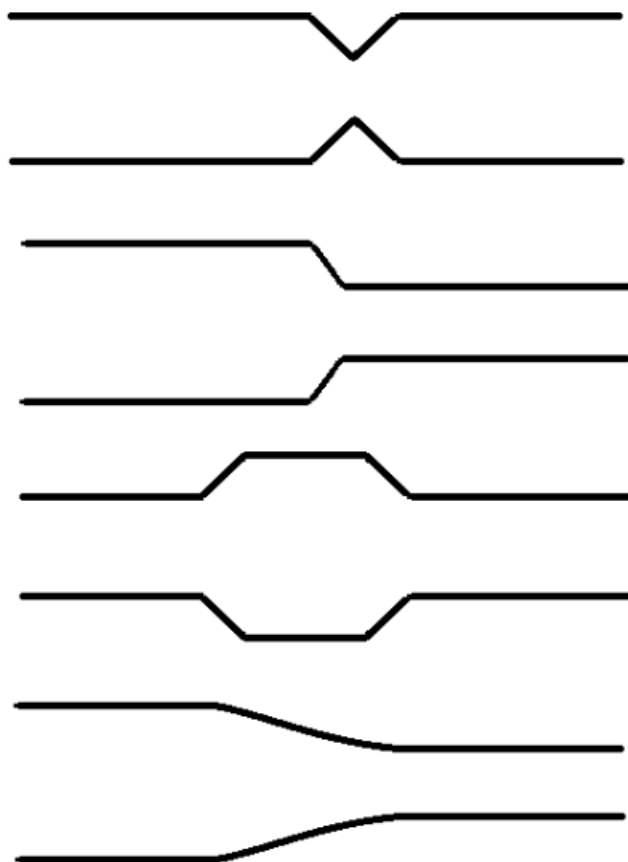
1-2 پردازش داده‌های حسگر سنجش فاصله

برای پردازش داده‌های به دست آمده از حسگر سنجش فاصله به اطلاعات و فرضیات اولیه ای نیاز داریم. طول اولیه فنر خودرو 34 سانتی‌متر در نظر گرفته شده است که می‌تواند بین اعداد 30 و 38 سانتی‌متر تغییر داشته باشد. در داده‌هایی که ایجاد کردیم زمان در بازه‌های یک دهم ثانیه افزایش یافته و مقدار طول فنر محاسبه و گزارش شده است. تغییر طول ناگهانی و به میزان یک سانتی‌متر فنر در محاسبات به عنوان داده ناهنجار (anomaly) تشخیص داده شده و به عنوان چاله تشخیص داده شده است.

داده‌ای که در دسترس داریم برای بازه زمانی 1000 ثانیه داده تولید کرده و همچنین باید این را در نظر داشته باشیم که زمانی که خودرو درون یک چاله می‌افتد تغییر طول فنر در بازه‌های زمانی بسیار کوچک ناچیز است لذا در نظر گرفتن 0.1 ثانیه عددی نسبتاً متعادل می‌باشد. همچنین چنانچه قرار بر پیاده سازی عملی این پروژه باشد میزان در نظر گرفتن بازه زمانی با توجه به داده‌های واقعی و نتایج به دست آمده این عدد باید تغییر یابد.

روش دوم این است که هر نقطه را که در نظر می‌گیریم. مقدار آن را با چندین نقطه قبل‌تر و بعدتر از آن مقایسه می‌کنیم و چنانچه تغییر بیش از 1 cm بود، آن نقطه را چاله در نظر می‌گیریم.

انواع چاله ها:



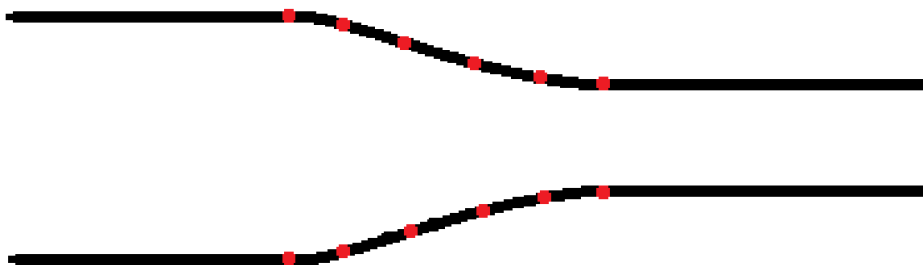
تصویر 1-2

تصویر 1-2 انواع مدل‌های چاله می‌باشد همانطور که در نمونه اول و دوم مشخص است در نقاط شکست سه بار این چاله در گزارش قید می‌شود زیرا برنامه چک می‌کند که این نقطه با نقطه بعدی اختلاف بیش از یک سانتی‌متر دارد. در میانه چاله برنامه چک می‌کند این نقطه با نقطه قبل و بعد یک سانتی‌متر اختلاف دارد و در پایان نیز چک می‌کند و نقطه با نقطه قبلی یک سانتی‌متر اختلاف دارد. باید این مورد را در نظر گرفت تا 3 رکورد تکراری ثبت شده را حذف کنیم.



تصویر 2-2

هنگامی که چاله فرمی شبیه تصویر 2-3 دارد، با توجه به بسامد نمونه برداری حسگر فاصله سنج اختلاف بین نمونه‌ها کمتر از یک سانتی‌متر می‌شود و ممکن است نقطه مورد نظر به عنوان چاله در نظر گرفته نشود لذا برای جلوگیری از این مشکل می‌توانیم هر نقطه را با دو یا چند نمونه قبل آن مقایسه کنیم. با این روش چنانچه بسامد دستگاه بسیار بالا باشد و رکوردهای متوالی تغییرات کمی داشته باشد می‌توانیم چاله‌ها را همچنان تشخیص دهیم و از نادیده گرفتن آن‌ها جلوگیری شود.



تصویر 3-2

حداقل تعداد رکوردهایی که نیاز است بررسی می‌کنیم تا از یافته شدن تمامی چاله‌ها اطمینان حاصل کنیم را می‌توان با آزمایش‌های مختلف و تعداد چاله‌های شناسایی شده به دست آوریم.

در تصویر 2-4 زیر نقاط روی خط، داده‌هایی است که حسگر اندازه‌گیری کرده و نقاط سبز داده‌هایی است که به عنوان چاله در نظر گرفته شده است.

داده‌های سبز رنگ به صورت تکراری این محدوده را به عنوان چاله اعلام کرده‌اند لذا نقاط همسایه را حذف می‌کنیم. نقاط آبی رنگ نقاطی اند که پس از پردازش به عنوان چاله در نظر گرفته شده است و نقاط مشکی رنگ داده‌ها حذف شده‌اند.



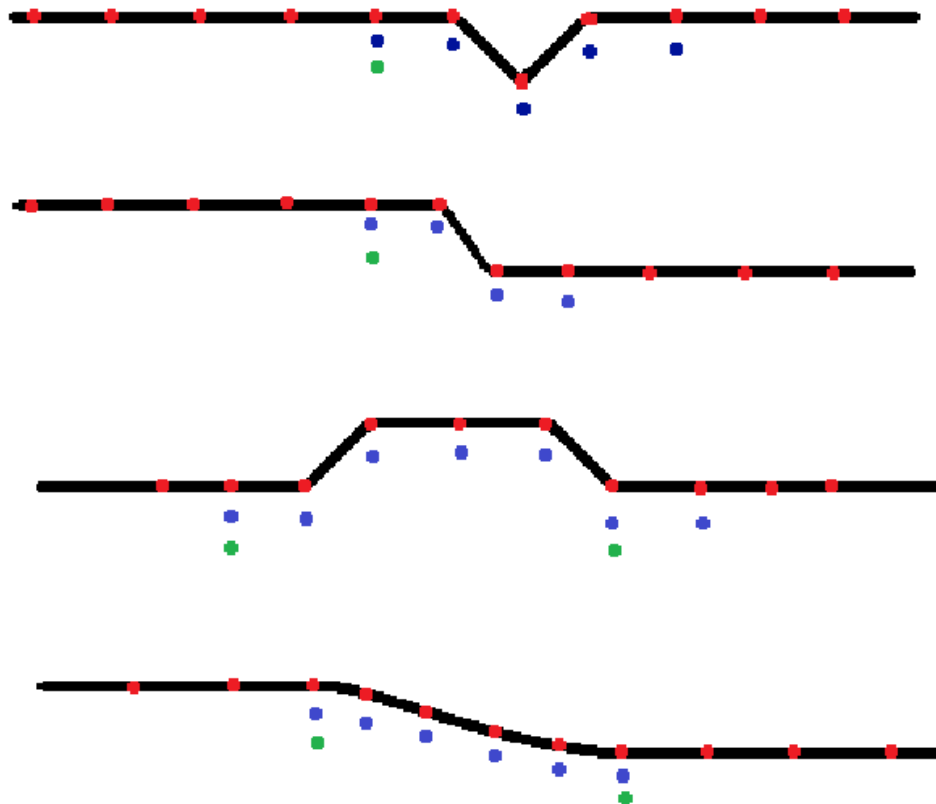
تصویر 2-4

روش حذف داده تکراری:

همانطور که در قسمت بالا اشاره شد برای یافتن برخی از چاله‌ها هر نقطه را با همسایگان دورتر نیز مقایسه می‌کنیم این عمل باعث می‌شد تا یک چاله در چندین رکورد ثبت شود برای همین زمانی که یک چاله شناسایی می‌شود تعداد 5 رکورد را بررسی می‌کنیم تا در محاسبات جدید چاله قبلی دوباره مورد بررسی قرار نگیرد.

در تصویر های 2-5 نقاط قرمز محل‌هایی است که حسگر داده ذخیره کرده و محل آبی نقاطی است که قبل از اعمال راه حل به عنوان چاله در نظر گرفته می‌شده و نقاط سبز محل‌هایی است که اکنون به عنوان چاله در نظر گرفته می‌شود

این راهکار تا حدود 70٪ حجم داده های تکراری را کاهش داده است .



تصویر 2-5

```

import pandas as pd
import matplotlib.pyplot as plt

def detect_and_save_special_records(file_path):
    # Load the Excel
    df = pd.read_excel(file_path)

    # Check if the required columns ('Time (s)' and 'Distance(cm)') exist
    if 'Time (s)' in df.columns and 'Distance(cm)' in df.columns:
        # List to store special records
        special_records = []

        # Sample 10% of the data
        sampled_df1 = df.sample(frac=0.1, random_state=42) # random_state
ensures reproducibility

        # Sort the sampled data by 'Time (s)' for better visualization
        sampled_df1 = sampled_df1.sort_values(by='Time (s)')

        # Plot the sampled data
        plt.figure(figsize=(10, 6))
        plt.plot(sampled_df1['Time (s)'], sampled_df1['Distance(cm)'],
marker='o', linestyle='-', color='b')
        plt.title('Time vs Distance (10% of Data)')
        plt.xlabel('Time (s)')
        plt.ylabel('Distance (cm)')
        plt.grid(True)
        plt.show()

        # Iterate through the DataFrame
        i = 2
        while i < len(df) - 2:
            curr_distance = df.loc[i, 'Distance(cm)']
            prev_distance_1 = df.loc[i - 1, 'Distance(cm)']
            prev_distance_2 = df.loc[i - 2, 'Distance(cm)']
            next_distance_1 = df.loc[i + 1, 'Distance(cm)']
            next_distance_2 = df.loc[i + 2, 'Distance(cm)']

            if (abs(curr_distance - prev_distance_1) >= 1 or
                abs(curr_distance - prev_distance_2) >= 1 or
                abs(curr_distance - next_distance_1) >= 1 or

```

```

        abs(curr_distance - next_distance_2) >= 1):
            special_records.append(df.loc[i])
            i += 5#if we found an hole skip two next record
            continue
        i += 1

    # Create a new DataFrame with the special records
    special_df = pd.DataFrame(special_records, columns=['Time (s)',
'Distance(cm)'])

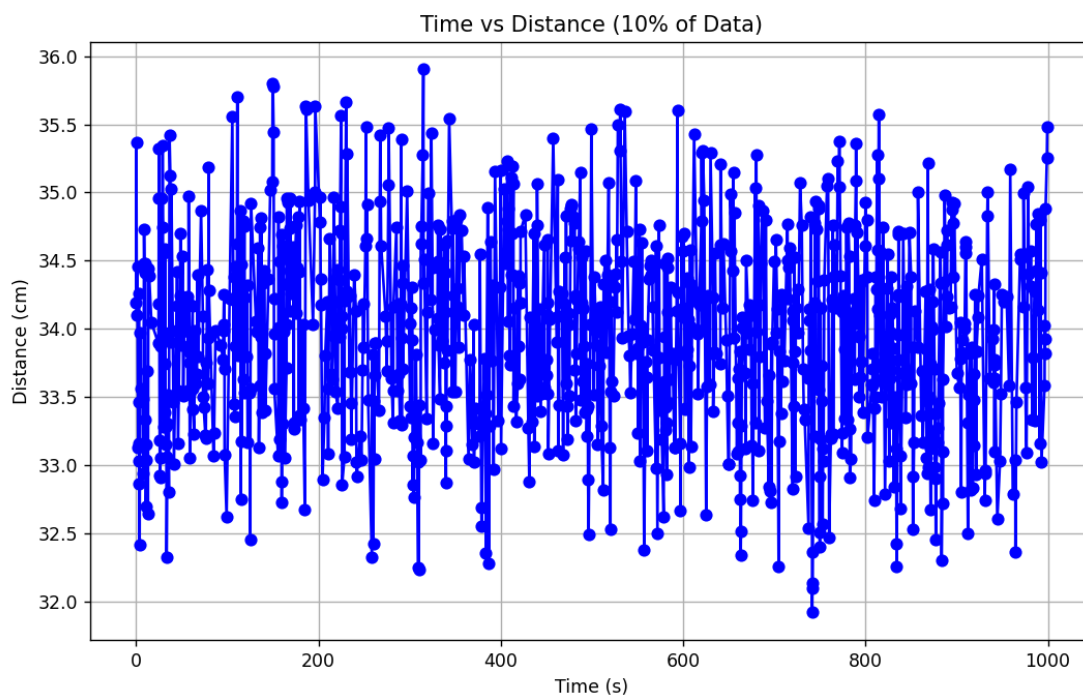
    # Save the special records
    special_output_file = 'special_records_2_prev_next.xlsx'
    special_df.to_excel(special_output_file, index=False)
    print(f"Special records have been saved to the file
'{special_output_file}'.")

    #Plot the special records
    plt.figure(figsize=(10, 6))
    plt.plot(special_df['Time (s)'], special_df['Distance(cm)'], marker='o',
linestyle='-', color='r')
    plt.title('Special Records: Distance Differing by 1 or More Units from 1
or 2 Previous or 1 or 2 Next Records')
    plt.xlabel('Time (s)')
    plt.ylabel('Distance (cm)')
    plt.grid(True)
    plt.show()
else:
    print("Error: The Excel file must contain 'Time (s)' and 'Distance(cm)'
columns.")

file_path = 'sensor_data1.xlsx'
detect_and_save_special_records(file_path)

```

نمایش نموداری ۱۰ درصد از داده‌ها (تعداد کل 10000)



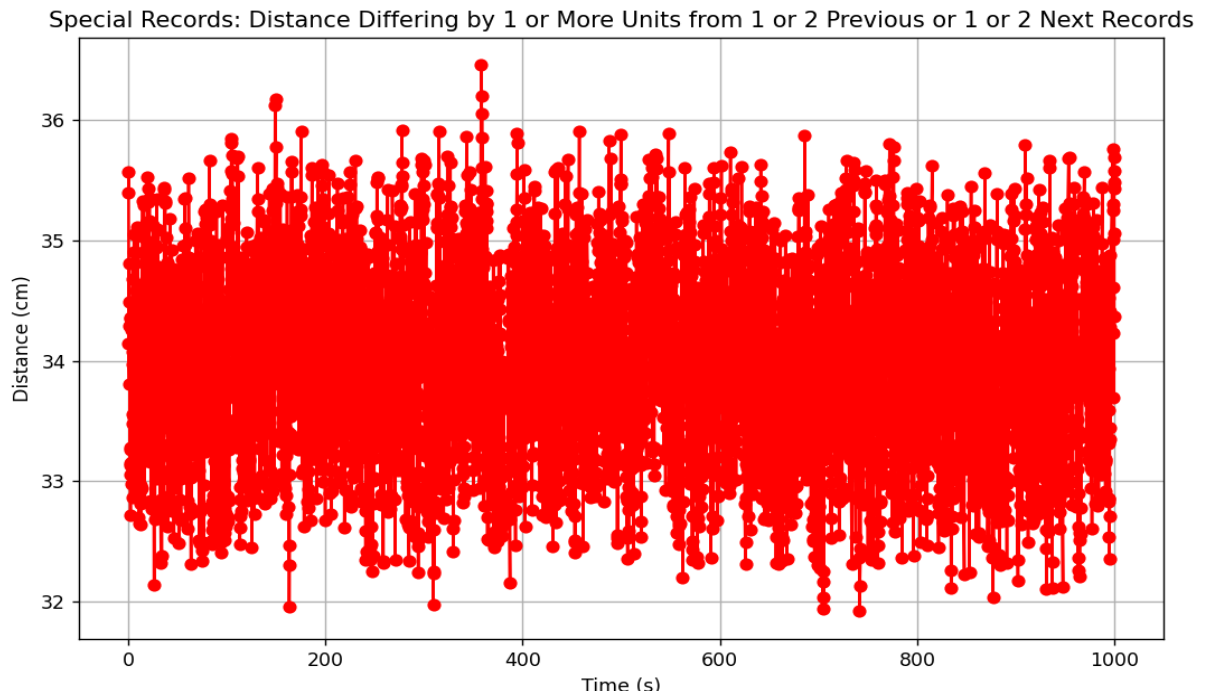
تصویر 6-2

40	3.8	32.21502	21	1.9	33.26523	A		B
41	3.9	32.41645	22	2	33.2817	1	Time (s)	istance(cm)
42	4	32.93608	23	2.1	33.05821	2	0	34.18979
43	4.1	32.81406	24	2.2	34.35848	3	0.1	34.68234
44	4.2	33.5612	25	2.3	33.1295	4	0.2	35.03239
45	4.3	32.94604	26	2.4	32.71773	5	0.3	35.36798
46	4.4	33.79814	27	2.5	34.34985	6	0.4	35.39447
47	4.5	32.86032	28	2.6	33.5258	7	0.5	35.5748
48	4.6	34.07754	29	2.7	34.44133	8	0.6	34.14556
49	4.7	33.97402	30	2.8	34.13143	9	0.7	35.02028
50	4.8	34.54468	31	2.9	33.46049	10	0.8	34.09921
51	4.9	34.67627	32	3	33.69378	11	0.9	34.73086
52	5	33.55782	33	3.1	34.25701	12	1	33.92522
53	5.1	33.47533	34	3.2	32.97755	13	1.1	33.80648
54	5.2	34.90345	35	3.3	33.15882	14	1.2	34.80959
55	5.3	33.27594	36	3.4	33.34024	15	1.3	34.24843
56	5.4	33.53323	37	3.5	33.02786	16	1.4	34.45365
57	5.5	33.52612	38	3.6	32.86444	17	1.5	33.80325
			39	3.7	32.38394	18	1.6	34.28708
						19	1.7	34.4943
						20	1.8	33.14915

تصویر 7-2

تعداد چاله‌های پیدا شده بدون استفاده از راه حل بالا

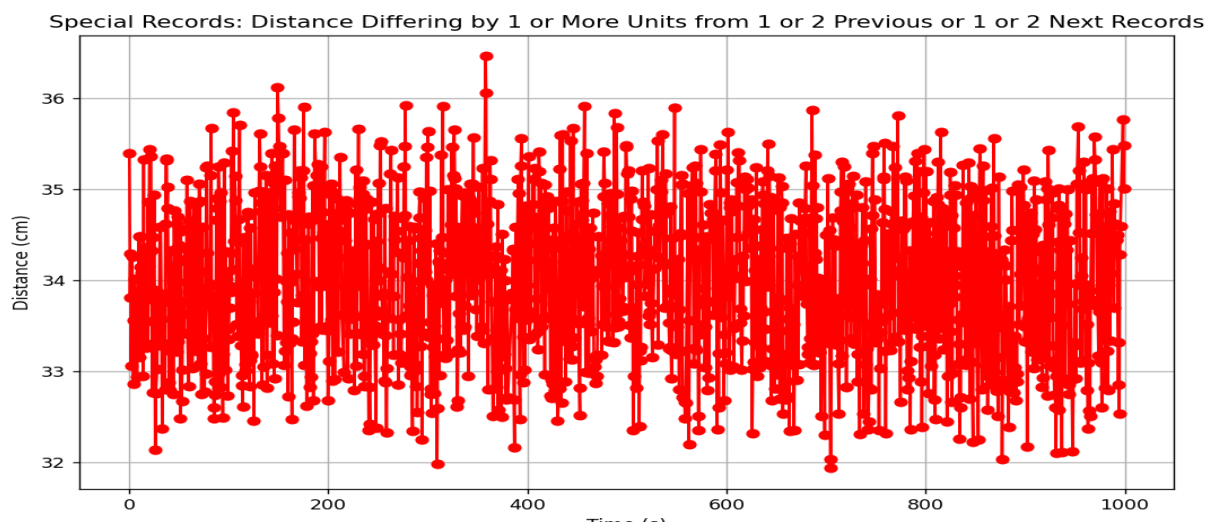
(دارای داده تکراری از یک چاله ، تعداد یافت شده 5155)



تصویر 8-2

تعداد چاله‌های پیدا شده با استفاده از راه حل بالا

(داده تکراری از هر چاله کاهش یافته، تعداد کل 1500)



تصویر 9-2

	A	B	C
1	Time (s)	distance(cm)	
2	0.4	35.39447	
3	1.1	33.80648	
4	1.6	34.28708	
5	2.1	33.05821	
6	3.1	34.25701	
7	4.5	32.86032	
8	5	33.55782	
9	5.9	34.01135	
10	6.9	33.59222	
11	8	32.9401	
12	8.5	33.33599	
13	9.5	33.87456	
14	10.1	34.48195	
15	10.6	33.15011	
16	11.1	34.47934	
17	11.6	34.14706	
18	12.4	34.04393	
19	12.9	34.0854	
20	13.6	33.53571	

تصویر 10-2

2-2 سنسور ضربه و ارتعاش

داده‌هایی که برای این سنسور در نظر گرفته‌ایم همانند قسمت قبل ۱۰ هزار رکورد دارد که شامل دو ویژگی زمان و لرزش می‌باشد.

اگر از حسگرهایی استفاده کنیم که مولفه شتاب در سه جهت x و y و z را به دست می‌آورند. مقدار مولفه z را که شتاب وارد بر جسم در راستای عمود است را بررسی می‌کنیم.

این سنسورها به طور معمول دارای یک ولوم برای کاهش و افزایش شدت حساسیت می‌باشند. پس می‌توان این نتیجه را گرفت که داده‌های حس شده از محیط در یک بازه خاص قرار دارد و چنانچه از یک میانگینی بالاتر باشد این نقاط به عنوان چاله در نظر گرفته می‌شود.

الگوریتم مناسب:

برای تشخیص آنومالی‌ها در یک مجموعه داده از روش Z-Score استفاده می‌کنیم. عملکرد این تابع به صورت زیر است:

توضیح الگوریتم

مقدار Z-score برای ستون "Vibration" محاسبه می‌شود. Z-score نشان می‌دهد که هر مقدار بر حسب انحراف معیار (σ) چقدر از میانگین (μ) فاصله دارد که با فرمول تصویر 11-2 محاسبه می‌گردد.

$$\frac{Vibration_i - \mu}{\sigma} = {}_iZ$$

تصویر 11-2

آستانه‌بندی (Thresholding): داده‌هایی را فیلتر می‌کند که مقدار مطلق Z-score آن‌ها از مقدار آستانه، بیشتر باشد.

سنسور ضربه و ارتعاش نسبت به سنسور عمق سنج یک سنسور جانبی محسوب می‌شود و نتایجی که به دست می‌آید دارای اهمیت نسبتاً کمتری از سنسور عمق سنج می‌باشد لذا میزان حساسیت برای یافتن چاله را بالا می‌بریم.

تأثیر	مقدار آستانه
تعداد بیشتری از ناهنجاری‌ها را شناسایی می‌کند، از جمله نوسانات جزئی.	1.5
رویکرد متعادل، ناهنجاری‌های عمده را شناسایی می‌کند.  (پیشنهادی)	2.0
فقط مقادیر بسیار نادر و دور از میانگین را به عنوان ناهنجاری تشخیص می‌دهد.	3.0

تصویر 12-2

آستانه = 1 ، Thresholding=1

در تصویر 2-13 نمونه‌ای از نحوه کارکرد Z-Score نمایش داده شده است.

Time (s)	Vibration
1	10
2	12
3	11
4	100
5	13
6	14

تصویر 13-2

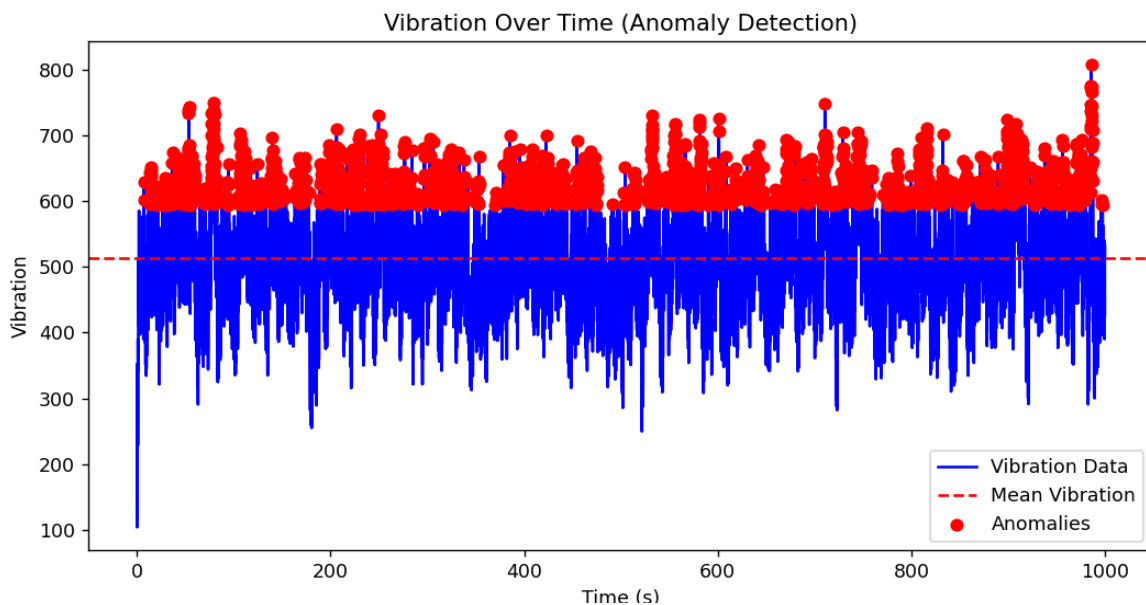
$(\sigma) = 34.9$ انحراف معیار ، $(\mu) = 26.67$ میانگین

Z-Score	مقدار ارتعاش
0.48-	10
0.42-	12
0.45-	11
2.1 ✓ (آنومالی در صورتی که آستانه 2.0 باشد)	100
0.40-	13
0.38-	14

تصویر 2-14

با توجه به توزیع داده‌ها مقادیر به دست آمده از Z-Score

مقدار 100 آنومالی تشخیص داده شده است.



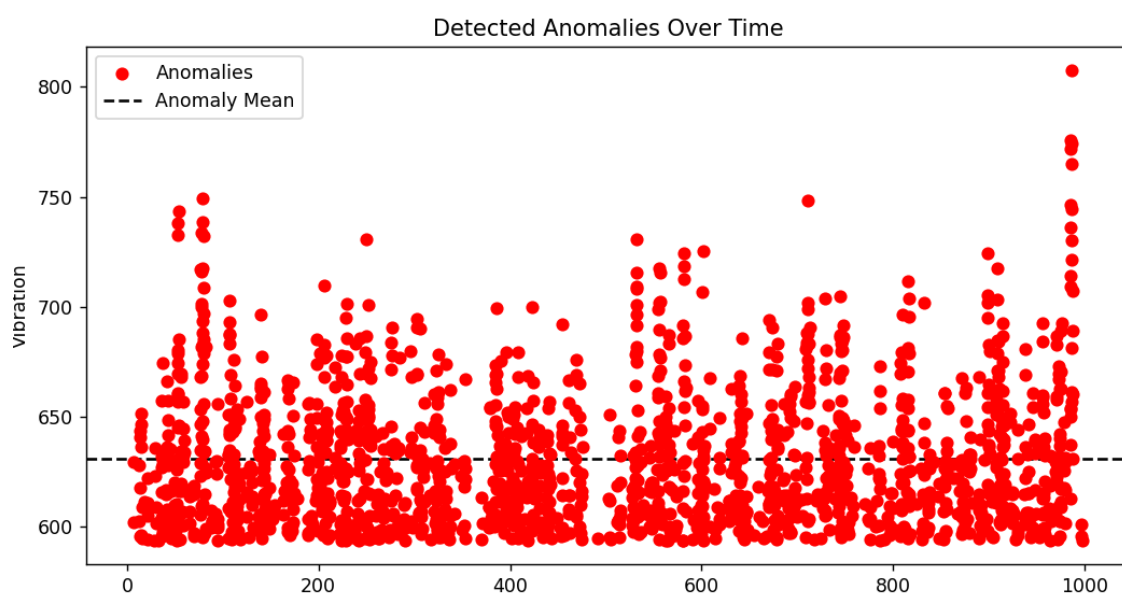
تصویر 2-15

همانطور که در تصویر 2-15 مشخص شده است نقاط قرمز رنگ به عنوان آنومالی بخش‌های آبی

رنگ تمام داده‌ها و قسمت خط چین میانگین می‌باشد

این موضوع را باید مورد توجه قرار دهیم. زمانی که مقدار از میانگین کمتر است نباید مورد نظر قرار

گیرد زیرا نشان دهنده کاهش لرزه بوده که در این نوع مسائل مورد بررسی و هدف ما نیست.



تصویر 2-16

در نمودار تصویر 2-16 داده‌هایی که به عنوان آنومالی تشخیص داده شدند را به صورت جداگانه

نمایش داده شده است که عمدتاً مقادیری بالای ۶۰۰ ثبت شده است.

می‌بایست خاطرنشان داشت با توجه به نوع سنسوری که در این مسئله به کار رفته مقادیر از بین 0

تا 1024 قابلیت تغییر داشتند و همچنین با توجه به ولومی که در این سنسور وجود دارد می‌توان

حساسیت آن را کاهش یا افزایش داد.

A	B	C	A	B	C
			Time (s)	Vibration	Z-Score
36.3	615.4412	1.275326	7.2	602.1923	1.109474
36.4	603.7245	1.128655	7.4	629.2156	1.447757
36.5	632.0307	1.482997	12.2	602.5948	1.114513
36.6	626.8034	1.41756	12.4	628.1465	1.434374
36.9	628.352	1.436946	13.3	595.5872	1.026791
37	674.4344	2.013814	13.7	643.5614	1.62734
37.1	597.4229	1.04977	13.9	640.5854	1.590085
37.3	602.1955	1.109514	14.1	617.8528	1.305515
37.5	630.3533	1.461999	14.2	636.2378	1.535662
37.6	601.03	1.094924	14.4	646.5223	1.664405
39.7	601.9631	1.106604	14.5	651.3293	1.72458
39.9	596.1273	1.033552	14.7	596.2903	1.035591
40.2	614.7723	1.266953	15.8	608.9257	1.193764
40.8	616.5589	1.289317	15.9	594.7416	1.016205
41.1	633.488	1.501239	18.4	611.2102	1.222362
41.3	608.078	1.183152	21	594.1492	1.008789
41.5	666.0972	1.909447	21.1	607.5021	1.175943
41.6	615.3754	1.274502	21.5	610.7358	1.216424
41.8	604.7114	1.141009	21.6	605.1344	1.146304
42	636.3036	1.536485			

تصویر 2-17

موضوعی که در سنسور عمق سنج مورد بررسی قرار دادیم این بود که رکوردهای تکراری ثبت نشود. هنگامی که داده های این سنسور را با سنسور عمق سنج ترکیب می کنیم داده هایی که از یک چاله ثبت شدند، از این دیتا نیز حذف می شوند.

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import zscore

# Load Excel
def load_data(file_path):
    return pd.read_excel(file_path)

def plot_data(df, anomalies):

    plt.figure(figsize=(10, 5))
    plt.plot(df["Time (s)"], df["Vibration"], label="Vibration Data",
color='blue')
    plt.axhline(df["Vibration"].mean(), color='red', linestyle='dashed',
label="Mean Vibration") # Mean Line
    plt.scatter(anomalies["Time (s)"], anomalies["Vibration"], color='red',
label="Anomalies", zorder=3) # Anomaly points
    plt.xlabel("Time (s)")
    plt.ylabel("Vibration")
    plt.title("Vibration Over Time (Anomaly Detection)")
    plt.legend()
    plt.show()

def plot_anomalies(anomalies):

    plt.figure(figsize=(10, 5))
    plt.scatter(anomalies["Time (s)"], anomalies["Vibration"], color='red',
label="Anomalies", zorder=3)
    plt.axhline(anomalies["Vibration"].mean(), color='black', linestyle='dashed',
label="Anomaly Mean")
    plt.xlabel("Time (s)")
    plt.ylabel("Vibration")
    plt.title("Detected Anomalies Over Time")
    plt.legend()
    plt.show()

def detect_anomalies(df, threshold=2.0):

    df["Z-Score"] = zscore(df["Vibration"]) # Compute Z-score
    mean_value = df["Vibration"].mean() # Compute mean
    return df[(df["Z-Score"] > threshold) & (df["Vibration"] > mean_value)] #
Filter anomalies above mean

```

```

def save_anomalies(anomalies, output_file="anomalies.xlsx"):

    #Save detected anomalies

    anomalies.to_excel(output_file, index=False)
    print(f"Anomaly records saved to '{output_file}'. Total anomalies:
{len(anomalies)}")

if __name__ == "__main__":
    file_path = "sensor_data2.xlsx"
    df = load_data(file_path)
    anomalies = detect_anomalies(df, threshold=1)

    plot_data(df, anomalies) # Plot full dataset
    plot_anomalies(anomalies) # Plot anomalies

    save_anomalies(anomalies)

```

2-3 پردازش داده‌های حسگر تشخیص صوت

با توجه به چالش‌هایی که در فصل گذشته در مورد این سنسور بیان شد، می‌توانیم از برخی سنسورها استفاده کنیم که تشخیص می‌دهند فرکانس دریافتی در چه بازه خاصی قرار دارد و تنها آن بازه را برای بررسی مورد توجه قرار می‌دهند.

مشکل دیگری ایجاد می‌شود که فرکانس صدایی که ایجاد می‌شود در اثر برخورد لاستیک خودرو با سطح آسفالت متفاوت از برخورد لاستیک خود را با سطح فلزی یا از جنس دیگر می‌باشد. همچنین شرایط جوی نیز در فرکانس صدای ایجاد شده تاثیر می‌گذارد. لذا در نظر گرفتن این موارد بار پردازشی را افزایش داده و همچنین داده‌ها دارای ضریبطمینان کمی می‌باشند.

سنسور تشخیص صوت که در فصل اول معرفی شد شدت صدا یا همان دامنه صدا را مورد بررسی قرار می‌دهد چنانچه دامنه صدا کوتاه‌تر باشد شدت آن کمتر و هرچه دامنه بلندتر شدت بیشتر است. اعداد با مقادیر بالا شدت بالای صدا و اعداد با مقادیر پایین شدت پایین صدا را نمایش می‌دهد.

در دید اولیه نسبت به داده‌های به دست آمده از این سنسور باید همانند سنسور لرزش عمل کنیم ولی با توجه به شرایط واقعی، همیشه در اثر برخورد لاستیک با جاده شدت صدا افزایش نمی‌یابد و هنگام افتادن لاستیک خودرو درون چاله برای بازه کوتاهی سطح تماس لاستیک با جاده کاهش می‌یابد و شدت صدای دریافت شده توسط سنسور نیز کاهش می‌یابد و سپس پس از برخورد مجدد شدت صدا به صورت ناگهانی افزایش می‌یابد. پس باید تغییرات ناگهانی را مورد بررسی قرار دهیم.

الگوریتم تشخیص آنومالی با استفاده از مدل ARIMA در داده‌های دنباله‌ای

ماشین بردار پشتیبان (ARIMA) یکی از مدل‌های معروف برای پیش‌بینی داده‌های سری زمانی است و می‌توان از آن برای تشخیص آنومالی در داده‌های دنباله‌ای استفاده کرد. این الگوریتم به طور خاص برای داده‌هایی که به صورت زمانی مرتب شده‌اند و تغییرات ناگهانی آن‌ها نیاز به شناسایی دارند، مناسب است.

مراحل الگوریتم:

1. مدل ARIMA را بر روی داده‌ها تطبیق می‌دهیم.
2. مقدارهای پیش‌بینی شده را با استفاده از مدل بدست می‌آوریم.
3. تفاوت بین مقادیر واقعی و پیش‌بینی شده مدل را محاسبه می‌کنیم.
4. آنومالی‌ها را شناسایی می‌کنیم (اگر باقیمانده بیشتر از آستانه‌ای که تعریف کرده‌اید باشد، آن نقطه به عنوان آنومالی شناسایی می‌شود.)

این روش سه پارامتر به عنوان ورودی می‌گیرد که با توجه به نوع مسئله ما و اینکه امکان دارد داده‌هایی که از محیط دریافت می‌کند با توجه به شرایط جاده‌ای متفاوت از یکدیگر باشند، تنظیمات را به نحوی انجام می‌دهیم که در شرایط مختلف نیز چاله‌ها را تشخیص دهد.

توضیح سه پارامتر الگوریتم ماشین بردار پشتیبان:

P : تعداد مقادیر قبلی که مدل برای پیش‌بینی مقدار جدید استفاده می‌کند.

مثال: اگر فروش روزهای قبل 100، 120، 130 باشد و $p=2$ باشد، مدل از دو مقدار قبلی (120 و 130) برای پیش‌بینی مقدار بعدی استفاده می‌کند.

D : پارامتر d در مدل ARIMA برای حذف روند (Trend) و ایستا کردن سری زمانی استفاده می‌شود.

چرا ایستا کردن داده‌ها مهم است؟

مدل ARIMA بر اساس وابستگی بین مقدارهای گذشته و آینده کار می‌کند. اگر داده‌ها روند داشته باشند (مثلاً همیشه در حال افزایش باشند)، مدل به جای یادگیری الگوهای واقعی، فقط افزایش مقدارها را یاد می‌گیرد و پیش‌بینی آن غیرقابل اعتماد می‌شود.

اگر d مقدار مناسبی داشته باشد: داده‌ها ایستا می‌شوند و مدل می‌تواند الگوهای واقعی را شناسایی کند.

اگر d کمتر از مقدار لازم باشد: سری زمانی همچنان دارای روند است و مدل پیش‌بینی درستی انجام نمی‌دهد.

اگر d بیشتر از مقدار لازم باشد: اطلاعات مفید از بین می‌رود و پیش‌بینی نادرست می‌شود.

فرض کنید داده‌های شما این گونه باشد: 2,5,10,17,26,37,50

این داده‌ها با سرعت متفاوتی افزایش پیدا می‌کنند (روند درجه دوم دارند).

اگر یک بار تفاضل‌گیری کنیم:

تفاضل‌گیری اول ($d=1$)

نتیجه: 3,5,7,9,11,13

هنوز روندی در داده‌ها وجود دارد! پس باید یک بار دیگر تفاضل بگیریم.

تفاضل‌گیری دوم ($d=2$)

نتیجه: 2,2,2,2,2

اکنون داده‌ها ایستا شده‌اند و مدل ARIMA می‌تواند آن‌ها را تحلیل کند.

Q: این پارامتر تعداد مقادیر قبلی از خطاهای پیش‌بینی را مشخص می‌کند که برای بهبود

پیش‌بینی‌های جدید استفاده می‌شود.

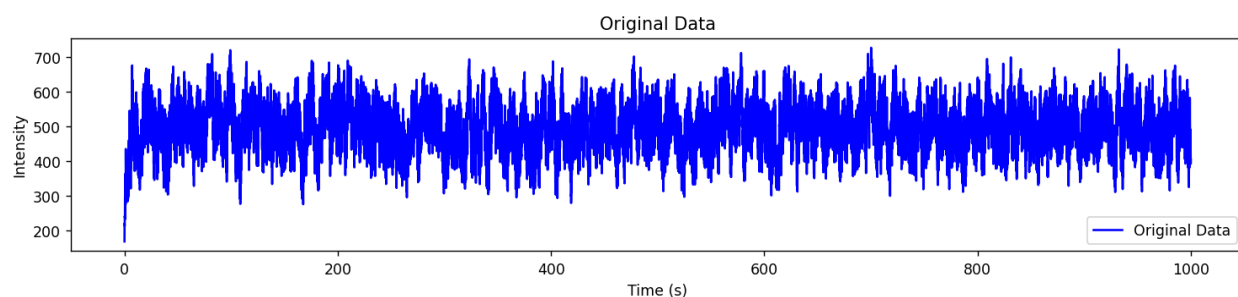
فرض کنید مدل مقدار بعدی را 160 پیش‌بینی کرده ولی مقدار واقعی 155 بوده است، یعنی

خطای 5- داریم.

اگر $q=1$ باشد، مدل این خطا را در پیش‌بینی‌های بعدی در نظر می‌گیرد تا دقت بیشتر شود.

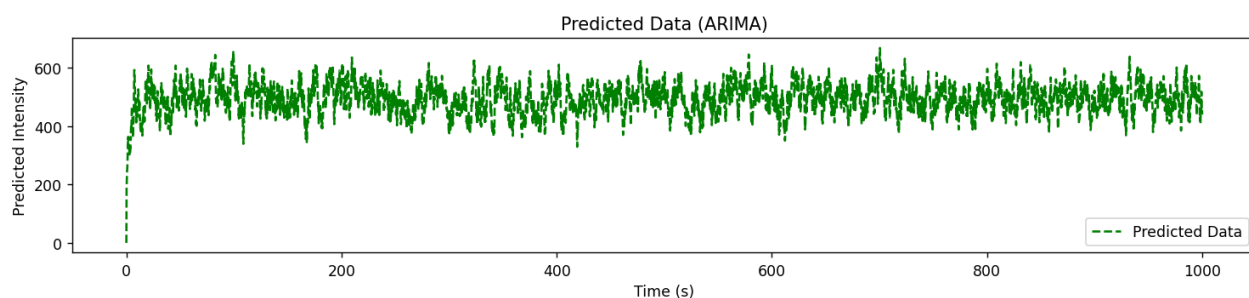
تحلیل داده ها:

در این تصویر 18-2 داده ها به صورت نموداری نمایش داده شده اند.



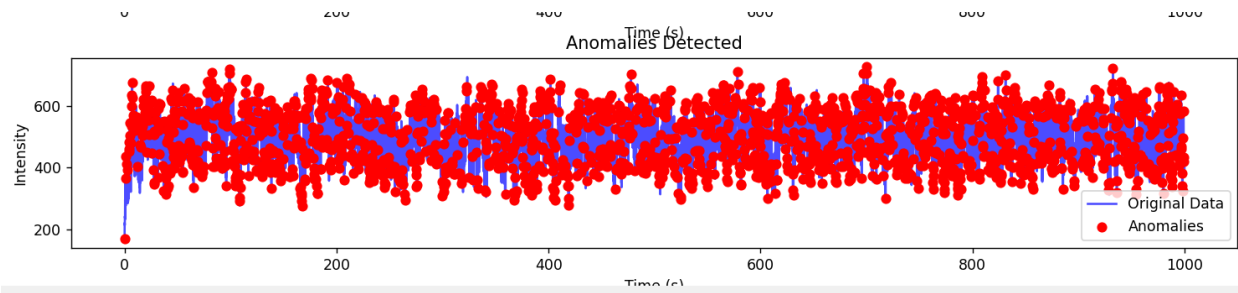
تصویر 18-2

تصویر 19-2 نمایش می دهد که مدل چه مقادیری را برای ما پیش بینی کرده است.



تصویر 19-2

با توجه به داده ها و مقادیر پیش بینی شده نقاط قرمز رنگ در تصویر 20-2 به عنوان آنومالی تشخیص داده شده است.



تصویر 20-2

A	B	C	D	E
Time (s)	Intensity	Predicted	Residuals	Anomaly
0	169.2146403	0	169.2146403	1
1	364.8457996	269.0745804	95.77121921	1
1.2	435.0182556	300.6159329	134.4023227	1
2.2	410.4975936	323.2054483	87.29214526	1
3	391.4736304	297.4454755	94.02815491	1
3.9	451.4938483	343.2483208	108.2455275	1
4.1	459.3505188	356.4282377	102.9222811	1
4.5	482.755768	392.8886094	89.86715862	1
4.7	503.8450934	397.5453323	106.2997611	1
4.8	544.9622645	422.8796752	122.0825893	1
6.1	477.9983555	360.6239157	117.3744399	1
6.7	533.0117852	423.7951834	109.2166017	1
6.8	547.1921055	451.5632096	95.6288959	1
6.9	598.3899578	481.0507604	117.3391974	1
7	633.34523	512.9788241	120.3664059	1
7.1	625.6906864	541.4997247	84.19096178	1
7.2	674.995915	562.1490299	112.8468851	1
8.4	567.5034201	450.2597231	117.243697	1
8.6	559.626497	463.4429069	96.18359013	1
9.4	518.7415608	425.5537876	93.18777317	1
10.3	521.9031229	432.9907782	88.91234476	1
10.5	564.0088713	453.2528673	110.7560039	1
11.7	564.4279399	454.3151146	110.1128253	1
11.9	550.4266824	459.4326189	90.99406352	1

تصویر 21-2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# Load the dataset
file_path = 'sensor_data3.xlsx'
df = pd.read_excel(file_path)

# Fit ARIMA model and detect anomalies
def detect_anomalies_arima(df, p=5, d=1, q=5, threshold_factor=1.4):
    # Fit model
    model = ARIMA(df['Intensity'], order=(p, d, q))
    model_fit = model.fit()

    # Get predicted values
    df['Predicted'] = model_fit.predict(start=0, end=len(df)-1, dynamic=False)

    # Calculate difference between actual and predicted values
    df['Residuals'] = df['Intensity'] - df['Predicted']

    # Use standard deviation of residuals as dynamic threshold
    std_residual = df['Residuals'].std()
    threshold = std_residual * threshold_factor

    # Identify anomalies based on dynamic threshold
    df['Anomaly'] = df['Residuals'].apply(lambda x: 1 if abs(x) > threshold else
0)

    anomalies = df[df['Anomaly'] == 1]

    return anomalies, df

# Detect anomalies
anomalies, df_with_predictions = detect_anomalies_arima(df)

# Display anomalies
#print("Detected Anomalies:")
#print(anomalies)

# Create subplots to display original data, predicted data, and anomalies
fig, axes = plt.subplots(3, 1, figsize=(12, 18))

# Plot the original data

```

```

axes[0].plot(df['Time (s)'], df['Intensity'], label="Original Data",
color='blue')
axes[0].set_title('\nOriginal Data')
axes[0].set_xlabel('Time (s)\n\n\n')
axes[0].set_ylabel('Intensity')
axes[0].legend()

# Plot the predicted data
axes[1].plot(df['Time (s)'], df['Predicted'], label="Predicted Data",
color='green', linestyle='--')
axes[1].set_title('\nPredicted Data (ARIMA)')
axes[1].set_xlabel('Time (s)\n\n')
axes[1].set_ylabel('Predicted Intensity')
axes[1].legend()

# Plot anomalies
axes[2].plot(df['Time (s)'], df['Intensity'], label="Original Data",
color='blue', alpha=0.7)
axes[2].scatter(anomalies['Time (s)'], anomalies['Intensity'], color='red',
label="Anomalies", zorder=5)
axes[2].set_title('\nAnomalies Detected')
axes[2].set_xlabel('Time (s)\n\n')
axes[2].set_ylabel('Intensity')
axes[2].legend()

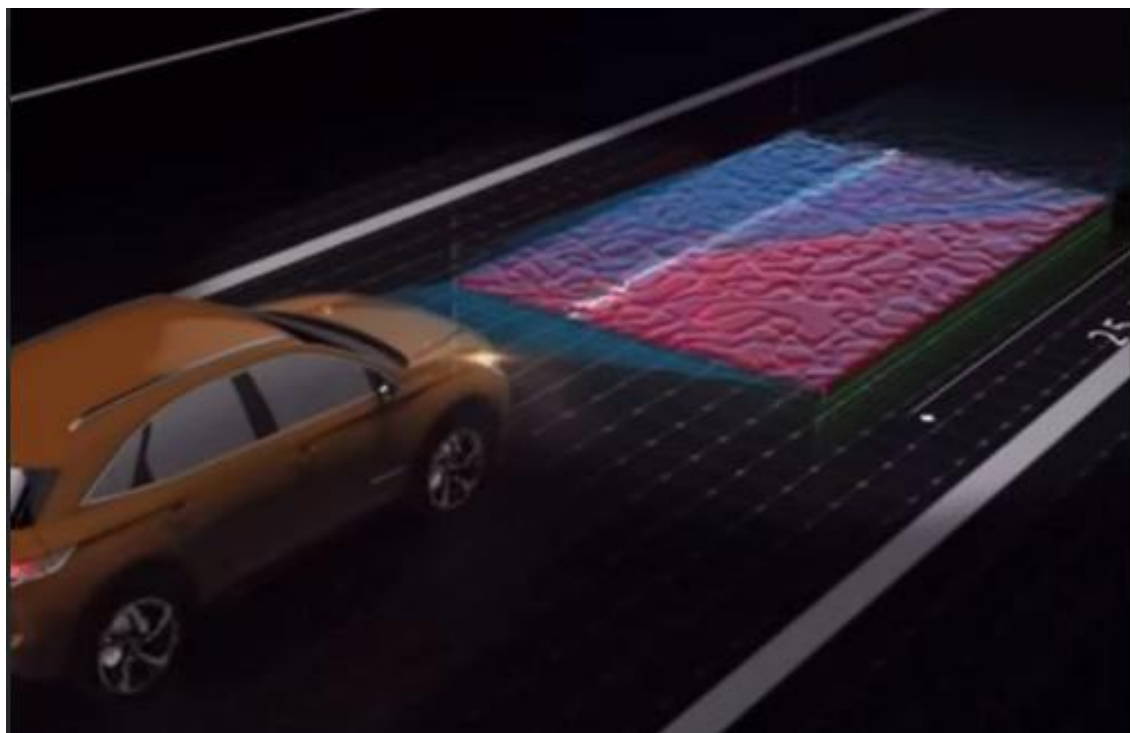
# Show the plot
plt.tight_layout()
plt.show()

# Save anomalies
anomalies.to_excel('detected_anomalies_1-3.xlsx', index=False)
print(f"Anomalies detected and saved to 'detected_anomalies1-3.xlsx'")

```

2-4 پردازش داده‌های تصویری

در تصاویری که توسط سنسور دوربین ثبت می‌شود تاریخ را به عنوان نام عکس در نظر می‌گیریم. همچنین بر روی مسیرهای شهری و بین شهری خطوط راهنمایی و رانندگی وجود دارد که سنسور دوربین می‌تواند آنها را به عنوان چاله تشخیص داده و دقت کاهش می‌یابد.



تصویر 2-22

روش‌های پردازش تصویری

می‌توانیم بافت تصاویر را به طور کلی در نظر بگیریم و قسمت‌هایی که تغییرات محسوس بود و یا بافت تغییر یافته بود را تشخیص دهیم.

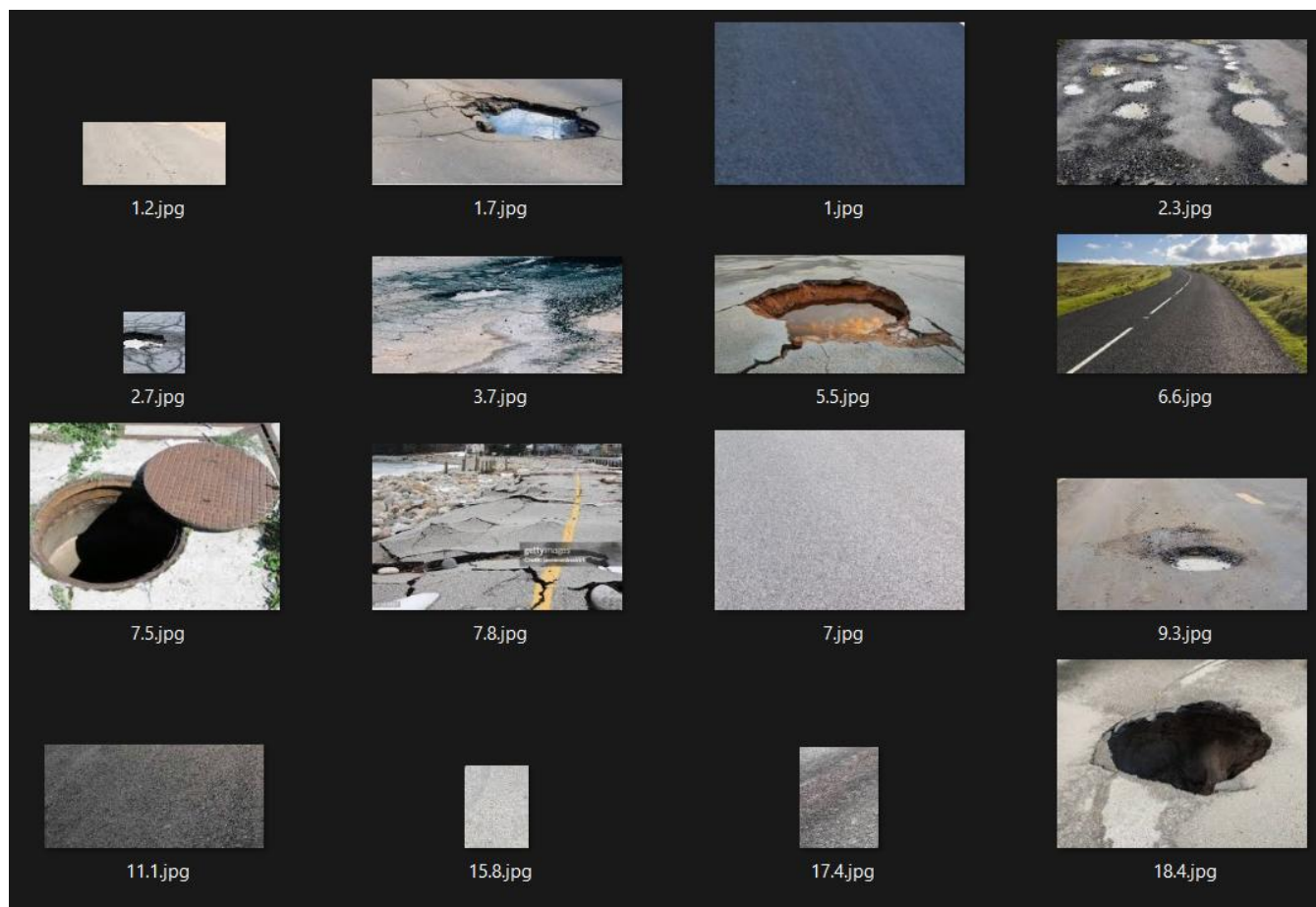
برای به دست آوردن بافت کلی تصویر از فیلترهای گرادیان جهت‌های افقی عمودی و همچنین تشخیص لبه استفاده می‌کنیم تا چنانچه بخش‌هایی از تصویر تغییر ناگهانی و شدید داشت تشخیص داده شود.

با توجه به محل نصب دوربین بازه‌ای که مورد تصویربرداری قرار می‌گیرد بالغ بر 2 متر مربع می‌باشد هدف تشخیص چاله‌هایی با ابعاد 20×25 و یا بیشتر می‌باشد پس تصویر را به 40 قسمت تقسیم می‌کنیم و میانگین رنگی هر قسمت را به دست می‌آوریم. قسمت‌هایی که میانگین رنگی با اختلاف بسیار بالا دارند را مشخص می‌کنیم.

تصاویری که از هر دو روش کاندیدای تصاویر ثبت چاله شدند را اشتراک می‌گیریم و نقاط باقیمانده را به عنوان نقاط هدف در نظر می‌گیریم.

در نهایت نتیجه را در یک فایل اکسل ذخیره می‌کنیم. به این صورت که نام عکس (زمان ثبت تصویر) texture, abnormal score در فایل قرار گرفته است.

دیتاست: تعداد عکس چاله 11 عدد است.



تصویر 2-23

نتایج روش اول) با توجه به تصویر 24-2 این روش 1 خطا دارد

	A	B	C	D
1	mage Name	texture Score	normal Ratio	
2	1.7.jpg	863.8619	0.05865	
3	18.4.jpg	652.5892	0.226512	
4	2.3.jpg	747.1045	0.157366	
5	2.7.jpg	1217.134	0.1224	
6	3.7.jpg	720.7919	0.143096	
7	5.5.jpg	836.5515	0.105306	
8	6.6.jpg	752.0792	0.263413	
9	7.5.jpg	998.894	0.482028	
10	7.8.jpg	1145.021	0.150303	
11				
12				

تصویر 24-2

نتایج روش دوم) با توجه به تصویر 25-2 این روش هیچ خطایی ندارد

	A	B	
1	mage Name		
2	1.7.jpg		
3	18.4.jpg		
4	2.3.jpg		
5	2.7.jpg		
6	3.7.jpg		
7	5.5.jpg		
8	6.6.jpg		
9	7.5.jpg		
10	7.8.jpg		
11	9.3.jpg		
12			

تصویر 25-2

نتایج نهایی در فایل merged_analysis1-4 ذخیره شده است.

```
import cv2
import numpy as np
import os
import pandas as pd

def calculate_mean_color(image):
    return np.mean(image, axis=(0, 1)) # Mean color (BGR) for each part

def analyze_color_by_parts(image, num_parts=40):
    h, w, _ = image.shape
    section_height = h // 8 # Dividing into 8 rows
    section_width = w // 5 # Dividing into 5 columns
    color_means = []

    for i in range(8):
        for j in range(5):
            section = image[i * section_height:(i + 1) * section_height, j *
section_width:(j + 1) * section_width]
            color_means.append(calculate_mean_color(section))

    color_means = np.array(color_means)
    std_color_values = np.std(color_means, axis=0) # Standard deviation of mean
colors
    return np.any(std_color_values > 15), color_means # Flag image if variation
is high

def calculate_texture_score(image):
    blurred = cv2.GaussianBlur(image, (5, 5), 0)
    grad_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=5)
    grad_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=5)
    grad_magnitude = np.sqrt(grad_x**2 + grad_y**2)
    return np.std(grad_magnitude) # Standard deviation of gradients

def detect_abnormal_regions(image):
    mean_brightness = np.mean(image)
    dark_threshold = mean_brightness * 0.5
    bright_threshold = mean_brightness * 1.5
    abnormal_ratio = (np.sum(image < dark_threshold) + np.sum(image >
bright_threshold)) / image.size
    return abnormal_ratio

def analyze_road_surface(image_path):
```

```

image = cv2.imread(image_path)
if image is None:
    return None, None, None

gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
texture_score = calculate_texture_score(gray_image)
abnormal_ratio = detect_abnormal_regions(gray_image)
color_variation, _ = analyze_color_by_parts(image)

return texture_score, abnormal_ratio, color_variation

def process_images(image_folder, texture_threshold=400,
abnormal_ratio_threshold=0.05):
    all_data = []
    special_data = []
    color_data = []

    for filename in os.listdir(image_folder):
        if filename.lower().endswith(('.png', '.jpg', '.jpeg')):
            image_path = os.path.join(image_folder, filename)
            texture_score, abnormal_ratio, color_variation =
analyze_road_surface(image_path)

            name, _ = os.path.splitext(filename) # Remove file extension

            if texture_score is not None:
                all_data.append([name, texture_score, abnormal_ratio])

            if texture_score is not None and texture_score > texture_threshold
and abnormal_ratio > abnormal_ratio_threshold:
                special_data.append([name, texture_score, abnormal_ratio])

            if color_variation:
                color_data.append([name])

    return all_data, special_data, color_data

def save_results(all_data, special_data, color_data, output_all, output_special,
output_color):
    if all_data:
        pd.DataFrame(all_data, columns=["Time (s)", "Texture Score", "Abnormal
Ratio"]).to_excel(output_all, index=False)
        print(f"All images' results saved in {output_all}.")

    if special_data:

```

```

        pd.DataFrame(special_data, columns=["Time (s)", "Texture Score",
"Abnormal Ratio"]).to_excel(output_special, index=False)
        print(f"Special images' results saved in {output_special}.")
    else:
        print("No special images met the criteria.")

    if color_data:
        pd.DataFrame(color_data, columns=["Time (s)"]).to_excel(output_color,
index=False)
        print(f"Color analysis results saved in {output_color}.")
    else:
        print("No images with significant color variation found.")

def merge_files(file1, file2, output_file="merged.xlsx"):
    df1 = pd.read_excel(file1)
    df2 = pd.read_excel(file2)
    merged_df = pd.merge(df1, df2, on="Time (s)", how="inner")
    merged_df.to_excel(output_file, index=False)
    print(f"Merged file saved as '{output_file}' with {len(merged_df)} matching
records.")

# Example usage:
image_folder = "road_images"
output_excel_all = "all_images_analysis1-4.xlsx"
output_excel_special = "special_images_analysis1-4.xlsx"
output_excel_color = "color_analysis1-4.xlsx"

all_data, special_data, color_data = process_images(image_folder)
save_results(all_data, special_data, color_data, output_excel_all,
output_excel_special, output_excel_color)

# Merge special and color analysis files
merge_files(output_excel_special, output_excel_color, "merged_analysis1-4.xlsx")

```

2-5 حسگر موقعیت مکانی

داده‌های اولیه مربوط به سنسور جی پی اس ایجاد شدند و دارای چند ستون می‌باشند که در قسمت زیر نمایش داده شده است.

NMEA GPGGA

Time (s)	Latitude	Longitude	Altitude (m)	Satellites	NMEA Sentence
----------	----------	-----------	--------------	------------	---------------

ستون اول زمان است که هر یک دهم ثانیه افزایش می‌یابد.

ستون دوم عرض جغرافیایی را نمایش می‌دهد که چنانچه E باشد مثبت و W باشد عدد منفی را نمایش می‌دهد.

ستون سوم طول جغرافیایی را نمایش می‌دهد که چنانچه N باشد مثبت و S باشد منفی است.

ستون چهارم ارتفاع از سطح دریا را نمایش می‌دهد و ستون پنجم تعداد ماهواره‌هایی که در دسترس را نمایش می‌دهد.

ستون NMEA Sentence داده کامل دریافت را نمایش می‌دهد.

در تصویر 2-26 نمونه داده های حسگر موقعیت مکانی را می توانیم مشاهده کنیم.

	A	B	C	D	E	F	G	H	I	J	K	L
1	Time (s)	Latitude	Longitude	Altitude (m)	Satellites	NMEA Sentence						
2	0	51.18069	6.494152	260.74	8	\$GPGGA,000000.0,5110.841,N,629.649,E,1,8,1.3,260.74,M,30.0,M,,*						
3	0.1	-64.6775	-123.179	159.23	7	\$GPGGA,000000.1,6440.650,S,12310.749,W,1,7,1.3,159.23,M,33.5,M,,*						
4	0.2	58.26487	29.59147	655.87	11	\$GPGGA,000000.2,5815.892,N,2935.488,E,1,11,1.1,655.87,M,21.1,M,,*						
5	0.3	-81.4567	-174.321	388.63	10	\$GPGGA,000000.3,8127.404,S,17419.260,W,1,10,1.2,388.63,M,38.3,M,,*						
6	0.4	-19.8585	61.25097	607.8	5	\$GPGGA,000000.4,1951.511,S,6115.058,E,1,5,1.5,607.8,M,44.9,M,,*						
7	0.5	-34.3502	96.75326	98.33	8	\$GPGGA,000000.5,3421.010,S,9645.195,E,1,8,1.5,98.33,M,45.0,M,,*						
8	0.6	2.089473	51.63932	640.39	5	\$GPGGA,000000.6,25.368,N,5138.359,E,1,5,1.4,640.39,M,31.2,M,,*						
9	0.7	-53.0232	-75.1273	360.14	6	\$GPGGA,000000.7,531.395,S,757.637,W,1,6,1.0,360.14,M,34.6,M,,*						
10	0.8	-31.8354	-25.0904	363.44	7	\$GPGGA,000000.7,3150.125,S,255.421,W,1,7,1.9,363.44,M,44.1,M,,*						
11	0.9	-57.8687	-24.816	456.91	7	\$GPGGA,000000.9,5752.123,S,2448.958,W,1,7,1.6,456.91,M,32.2,M,,*						
12	1	5.264931	152.9056	919.52	6	\$GPGGA,000000.9,515.896,N,15254.333,E,1,6,0.6,919.52,M,22.0,M,,*						
13	1.1	18.83345	107.8999	323.31	9	\$GPGGA,000001.0,1850.007,N,10753.992,E,1,9,1.4,323.31,M,40.8,M,,*						
14	1.2	-45.1838	67.23652	89.61	6	\$GPGGA,000001.2,4511.029,S,6714.191,E,1,6,1.0,89.61,M,30.7,M,,*						
15	1.3	9.540391	-19.6638	326.3	6	\$GPGGA,000001.3,932.423,N,1939.830,W,1,6,1.3,326.3,M,35.2,M,,*						
16	1.4	-31.7844	-48.5994	12.65	7	\$GPGGA,000001.4,3147.062,S,4835.962,W,1,7,1.3,12.65,M,33.0,M,,*						
17	1.5	35.20446	46.87307	906.37	4	\$GPGGA,000001.5,3512.268,N,4652.384,E,1,4,1.8,906.37,M,39.3,M,,*						
18	1.6	-35.8319	-161.856	94.35	4	\$GPGGA,000001.6,3549.916,S,16151.377,W,1,4,0.5,94.35,M,40.4,M,,*						
19	1.7	-29.282	-85.712	908.92	9	\$GPGGA,000001.7,2916.922,S,8542.718,W,1,9,1.1,908.92,M,36.9,M,,*						
20	1.8	33.84937	-19.9957	268.04	6	\$GPGGA,000001.8,3350.962,N,1959.741,W,1,6,1.8,268.04,M,46.0,M,,*						
21	1.9	-46.9402	60.77002	336.1	11	\$GPGGA,000001.9,4656.410,S,6046.201,E,1,11,1.7,336.1,M,37.3,M,,*						
22	2	-50.0727	144.6462	182.23	11	\$GPGGA,000002.0,504.363,S,14438.773,E,1,11,1.1,182.23,M,38.6,M,,*						
23	2.1	39.23106	-3.23992	18.14	4	\$GPGGA,000002.1,3913.863,N,314.395,W,1,4,1.5,18.14,M,31.9,M,,*						
24	2.2	-35.0525	64.00017	246.94	12	\$GPGGA,000002.2,353.149,S,640.010,E,1,12,1.2,246.94,M,26.2,M,,*						
25	2.3	-18.1102	53.73538	555.5	10	\$GPGGA,000002.3,186.610,S,5344.123,E,1,10,1.2,555.5,M,36.6,M,,*						
26	2.4	-49.1716	-117.702	101.84	5	\$GPGGA,000002.4,4910.294,S,11742.105,W,1,5,1.5,101.84,M,27.6,M,,*						

تصویر 2-26

```
import random
import pandas as pd

# Generate NMEA GPGGA Sentence
def generate_gpgga_sentence(time, lat, lon, altitude, satellites):

    #Parameters:
    #    time (float): Time in seconds.
    #    lat (float): Latitude in decimal degrees.
    #    lon (float): Longitude in decimal degrees.
    #    altitude (float): Altitude in meters.
    #    satellites (int): Number of satellites used.
```

```

#Returns:
#    str: A formatted NMEA GPGGA sentence.

time_str = f"{int(time // 3600):02}{int((time % 3600) // 60):02}{int(time %
60):02}.{int((time * 10) % 10)}"
lat_deg = int(abs(lat))
lat_min = (abs(lat) - lat_deg) * 60
lat_dir = "N" if lat >= 0 else "S"
lon_deg = int(abs(lon))
lon_min = (abs(lon) - lon_deg) * 60
lon_dir = "E" if lon >= 0 else "W"
hdop = round(random.uniform(0.5, 2.0), 1)
geoid_sep = round(random.uniform(20, 50), 1)
return
f"$GPGGA,{time_str},{lat_deg}{lat_min:.3f},{lat_dir},{lon_deg}{lon_min:.3f},{lon_
dir},1,{satellites},{hdop},{altitude},M,{geoid_sep},M,,"

# Generate data
data = []
start_time = 0 # Start time in seconds
time_interval = 0.1 # Time interval between records

for i in range(10000):
    latitude = round(random.uniform(-90, 90), 6)
    longitude = round(random.uniform(-180, 180), 6)
    altitude = round(random.uniform(0, 1000), 2)
    satellites = random.randint(4, 12) # Number of satellites (4 to 12 typical
for GPS)
    gpgga_sentence = generate_gpgga_sentence(start_time, latitude, longitude,
altitude, satellites)

    data.append((start_time, latitude, longitude, altitude, satellites,
gpgga_sentence))
    start_time += time_interval # Increment time

# Create DataFrame
df = pd.DataFrame(data, columns=['Time (s)', 'Latitude', 'Longitude', 'Altitude
(m)', 'Satellites', 'NMEA Sentence'])

# Save to Excel
output_file = 'gps_gpgga_data5.xlsx'
df.to_excel(output_file, index=False)

print(f"Data has been saved to '{output_file}' with 10,000 sequential time
records.")

```

2-6 امتیاز دهی و ترکیب داده ها

هر کدام از سنسورها به صورت جداگانه نقاطی را به عنوان چاله تشخیص داده‌اند.

ارزش این اطلاعات باتوجه به نوع سنسور و ضریب اطمینان از تشخیص درست، تغییر می کند.

لذا به داده‌های دریافت شده از هر سنسور یک ضریب اهمیت تخصیص می‌دهیم تا در مجموع نقاطی که ضریب اهمیت بالاتری دارند را مهم تر در نظر بگیریم.

به عنوان مثال داده‌های به دست آمده از سنسور عمق سنج دارای صحت و اهمیت بالایی است لذا ضریب نسبتا بالایی به آن تعلق می‌گیرد.

ضریب داده‌های حسگر سنجش فاصله ۹، ضریب حسگر ضربه و ارتعاش ۳، ضریب حسگر تشخیص صوت ۲، ضریب دوربین تشخیص دهنده عوارض ۳ می‌باشد.

به `special_records_1-1` و `anomalies_1-2` و `detected_anomalies_1-3` و `merged_analysis1-4` یک ستون ضریب اضافه می کنیم.

تمام داده‌ها را در یک فایل خیره می‌کنیم و رکوردهایی که زمان یکسانی دارد را امتیاز آنها را نیز با هم جمع می‌کنیم.

نقاطی که امتیاز بیشتر از یا مساوی ۹ داشته باشند را نگه می‌داریم و مابقی را حذف می‌کنیم.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Time (s)	Score	Latitude	Longitude	Altitude (m)	Satellites	IEA Sentence						
2	304.8	14	81.2754	-34.1259	201.06	9	\$GPGGA,000504.8,8116.524,N,347.552,W,1,9,1.5,201.06,M,41.8,M,,*						
3	302.6	14	10.18017	-86.1732	603.43	7	\$GPGGA,000502.6,1010.810,N,8610.393,W,1,7,1.8,603.43,M,25.0,M,,*						
4	955.9	14	32.8327	-136.841	199.04	9	\$GPGGA,001555.9,3249.962,N,13650.467,W,1,9,1.3,199.04,M,40.2,M,,*						
5	936.8	14	87.54143	-92.0914	198.57	9	\$GPGGA,001536.8,8732.486,N,925.486,W,1,9,1.0,198.57,M,29.4,M,,*						
6	385.9	14	48.09646	13.06483	359.29	8	\$GPGGA,000625.9,485.788,N,133.890,E,1,8,1.5,359.29,M,25.8,M,,*						
7	384.9	14	-64.2552	41.18788	416.19	7	\$GPGGA,000624.9,6415.309,S,4111.273,E,1,7,0.6,416.19,M,24.2,M,,*						
8	394.7	14	-13.479	158.2575	800.25	11	\$GPGGA,000634.7,1328.741,S,15815.453,E,1,11,1.5,800.25,M,38.8,M,,*						
9	390.8	14	35.19305	96.93324	670.21	10	\$GPGGA,000630.8,3511.583,N,9655.994,E,1,10,1.0,670.21,M,44.2,M,,*						
10	284	14	-20.8007	45.14545	890.21	11	\$GPGGA,000443.9,2048.043,S,458.727,E,1,11,1.0,890.21,M,46.6,M,,*						
11	423.8	14	-79.8151	-45.0503	154.64	10	\$GPGGA,000703.8,7948.906,S,453.017,W,1,10,0.8,154.64,M,40.5,M,,*						
12	289.3	14	-6.50663	30.65434	582.93	6	\$GPGGA,000449.2,630.398,S,3039.260,E,1,6,0.7,582.93,M,35.3,M,,*						
13	462.7	14	-54.1228	-83.435	805.39	8	\$GPGGA,000742.7,547.370,S,8326.103,W,1,8,0.7,805.39,M,42.6,M,,*						
14	454.6	14	-4.05856	-3.32754	84.35	8	\$GPGGA,000734.6,43.514,S,319.652,W,1,8,0.8,84.35,M,41.5,M,,*						
15	251.1	14	-57.0456	23.31416	865.39	12	\$GPGGA,000411.0,572.738,S,2318.850,E,1,12,0.8,865.39,M,42.7,M,,*						
16	248.8	14	-85.5023	-178.005	428.77	9	\$GPGGA,000408.7,8530.135,S,1780.286,W,1,9,1.6,428.77,M,44.0,M,,*						
17	229.7	14	56.23669	12.26109	933.84	6	\$GPGGA,000349.6,5614.201,N,1215.665,E,1,6,1.2,933.84,M,31.1,M,,*						
18	228.6	14	68.21827	9.857709	999.52	6	\$GPGGA,000348.5,6813.096,N,951.463,E,1,6,1.6,999.52,M,22.1,M,,*						
19	216.9	14	15.62607	133.9004	670.02	7	\$GPGGA,000336.8,1537.564,N,13354.022,E,1,7,1.2,670.02,M,38.2,M,,*						
20	41.8	14	-58.5072	-107.208	621.1	8	\$GPGGA,000041.8,5830.430,S,10712.488,W,1,8,0.6,621.1,M,40.9,M,,*						
21	532.9	14	-34.9159	-101.488	225.3	7	\$GPGGA,000852.9,3454.955,S,10129.276,W,1,7,1.6,225.3,M,44.4,M,,*						
22	206	14	82.11676	-139.486	430.51	12	\$GPGGA,000325.9,827.006,N,13929.189,W,1,12,1.4,430.51,M,37.9,M,,*						
23	555.3	14	77.04681	157.9217	300.69	7	\$GPGGA,000915.3,772.808,N,15755.304,E,1,7,1.5,300.69,M,49.5,M,,*						
24	566	14	-61.7768	-118.869	113.55	12	\$GPGGA,000926.0,6146.609,S,11852.140,W,1,12,0.7,113.55,M,41.6,M,,*						
25	581.2	14	-50.8699	162.4847	507.47	12	\$GPGGA,000941.2,5052.193,S,16229.083,E,1,12,1.9,507.47,M,42.5,M,,*						
26	173.4	14	-15.5709	85.38577	598.94	12	\$GPGGA,000253.3,1534.255,S,8523.146,E,1,12,1.0,598.94,M,40.3,M,,*						
27	618.1	14	-32.4758	143.5609	480.09	5	\$GPGGA,001018.1,3228.549,S,14333.653,E,1,5,0.8,480.09,M,26.0,M,,*						
28	670.7	14	51.06417	150.4835	468.29	4	\$GPGGA,001110.7,513.850,N,15029.010,E,1,4,1.9,468.29,M,21.4,M,,*						
29	169.4	14	-18.1252	11.24729	287.12	10	\$GPGGA,000249.3,187.514,S,1114.838,E,1,10,1.2,287.12,M,38.1,M,,*						
30	646.5	14	-71.3521	9.504991	514.7	11	\$GPGGA,001046.5,7121.127,S,930.299,E,1,11,1.7,514.7,M,29.3,M,,*						
31	690.6	14	-74.3025	92.47399	886.7	5	\$GPGGA,001130.6,7418.151,S,9228.439,E,1,5,1.9,886.7,M,36.6,M,,*						
32	143.2	14	-31.718	-139.072	346.84	10	\$GPGGA,000223.1,3143.079,S,1394.346,W,1,10,1.0,346.84,M,33.4,M,,*						
33	693.2	14	-63.6876	-128.406	776.93	7	\$GPGGA,001133.2,6341.259,S,12824.381,W,1,7,1.0,776.93,M,40.1,M,,*						
34	748.2	14	-16.1011	171.103	704.63	11	\$GPGGA,001228.2,166.069,S,1716.178,E,1,11,0.6,704.63,M,36.3,M,,*						

تصویر 27-2

در این تصویر 27-2 تعدادی از چاله‌های شناسایی شده با همراه موقعیت مکانی آنها و امتیاز آنها قرار گرفته شده است که برای مثال رکورد اول نمایش دهنده این است که چاله توسط سنسور عمق سنج، صوت و لرزه سنج شناسایی شده است.

final_data_with_gps_data.xlsx فایل نهایی است.

```

import pandas as pd

def add_score_column(file_path, score, output_file):
    """
    Load an Excel file, retain only 'Time (s)', add a 'Score' column,
    and save the modified file.
    """
    df = pd.read_excel(file_path) # Load Excel
    df = df[['Time (s)']] # Keep only 'Time (s)' column
    df['Score'] = score # Add the 'Score' column
    df.to_excel(output_file, index=False) # Save the modified DataFrame
    print(f"Updated file saved as: {output_file}")

def merge_and_update_scores(file_paths, output_file):
    """
    Merge all files on 'Time (s)', sum the scores for matching values,
    remove records with scores fewer than 9, and save the final records.
    """
    dfs = {file: pd.read_excel(file) for file in file_paths} # Read all Excel
files

    # Start with the first file
    merged_df = dfs[file_paths[0]]

    # Merge with other files on 'Time (s)' and sum 'Score'
    for file in file_paths[1:]:
        df = dfs[file]
        merged_df = merged_df.merge(df, on='Time (s)', how='outer', suffixes=('',
'_drop'))

        # Sum score columns and update the main 'Score' column
        score_columns = [col for col in merged_df.columns if "Score" in col]
        merged_df['Score'] = merged_df[score_columns].sum(axis=1)
        merged_df.drop(columns=[col for col in score_columns if col != 'Score'],
inplace=True)

    # Remove records where 'Score' is fewer than 9
    merged_df = merged_df[merged_df['Score'] >= 9]

    # Sort by 'Score' in descending order
    merged_df = merged_df.sort_values(by='Score', ascending=False)

    merged_df.to_excel(output_file, index=False)
    print(f"Final merged file saved as: {output_file}")

```

```

def merge_with_gps_data(final_file, gps_file,
output_file="final_data_with_gps_data.xlsx"):

    # Merge the final merged data file with gps data on 'Time (s)' and overwrite
    matching records.

    # Load both datasets
    final_df = pd.read_excel(final_file)
    gps_df = pd.read_excel(gps_file)

    # Merge on 'Time (s)' with an inner join (only matching 'Time (s)' records
    will be kept)
    merged_df = pd.merge(final_df, gps_df, on="Time (s)", how="inner",
    suffixes=('_final', '_gps'))

    # Save the merged data
    merged_df.to_excel(output_file, index=False)
    print(f"Merged file saved as '{output_file}' with {len(merged_df)} matching
    records.")

def process_files_and_merge(files_scores, final_output_file, gps_file):

    #Process a dictionary of files, add scores, merge the final results,
    #and merge the GPS data.

    updated_files = []
    for file, (score, output_file) in files_scores.items():
        add_score_column(file, score, output_file)
        updated_files.append(output_file)

    # Merge and update scores
    merge_and_update_scores(updated_files, final_output_file)

    # Merge with GPS data
    merge_with_gps_data(final_output_file, gps_file)

# File paths and corresponding scores
files_scores = {
    "special_records_1-1.xlsx": (9, "special_records_1-1_updated.xlsx"),
    "anomalies_1-2.xlsx": (3, "anomalies_1-2_updated.xlsx"),
    "detected_anomalies_1-3.xlsx": (2, "detected_anomalies_1-3_updated.xlsx"),
    "merged_analysis1-4.xlsx": (3, "merged_analysis1-4_updated.xlsx")
}

```

```
# Final output file
final_output_file = "merged_sum_score.xlsx"

# GPS file
gps_file = "gps_gpgga_data5.xlsx"

# Process and merge the files with GPS data
process_files_and_merge(files_scores, final_output_file, gps_file)
```

فصل سوم

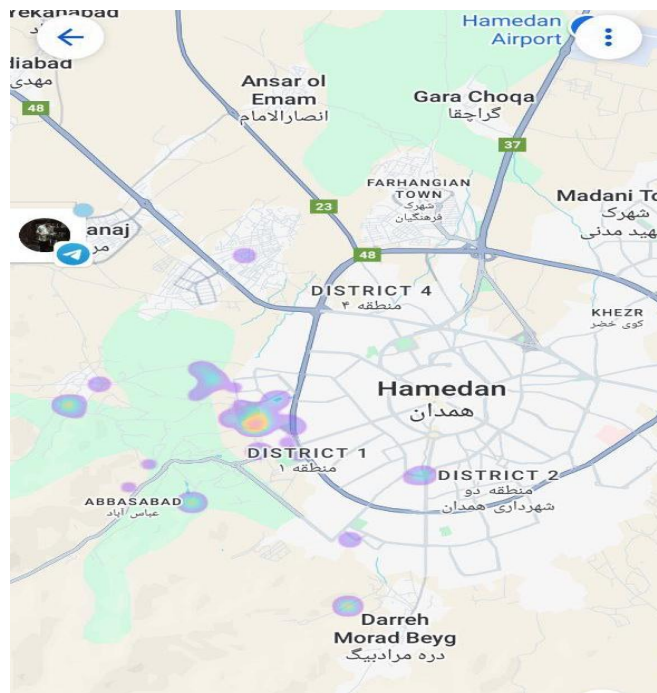
چارچوب نرم‌افزاری

در این بخش توضیحاتی در مورد فاز نرم‌افزاری و تحت وب پروژه داده شده است و چگونگی کارکرد آن نیز بیان شده.

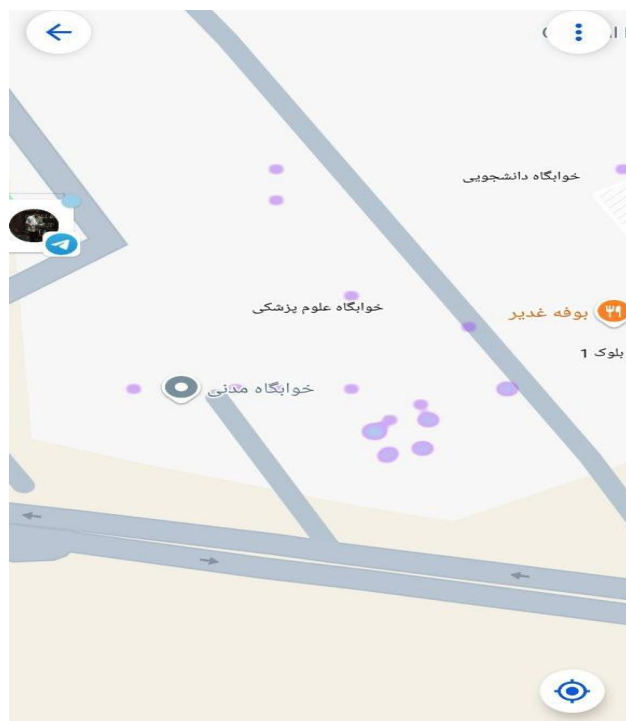
با ایجاد یک نسخه تحت وب یا نرم‌افزار اندروید می‌توانیم دسترسی‌هایی برای کاربران و کارفرما ایجاد کنیم.

یکی از قابلیت‌های برنامه، ثبت چاله‌ها توسط کاربران است که در این حالت موقعیت مکانی و حالت چاله (شامل چندین حالت مختلف که عدد یک کمترین اهمیت و عدد ۵ بیشترین اهمیت) و جزئیات چاله توسط کاربر وارد می‌شود.

قابلیت دیگر، نمایش تراکم چاله‌ها در نقشه است که به وسیله نقاط بر روی نقشه نمایش داده می‌شود که بزرگ و کوچک و رنگ نقاط تراکم آن را نمایش می‌دهد به صورتی که هرچه آن مناطق رنگ تیره‌تری داشته باشد تعداد چاله ثبت شده در آن منطقه زیادتر است و هر میزان که نقطه بزرگتر باشد تعداد چاله‌های ثبت شده در آن نقطه بیشتر است.



تصویر 1-3



تصویر 2-3

تصاویر 1-3 و 2-3 مفهوم مشابهی با آنچه گفته شد، نمایش می‌دهند به صورتی که نقاطی را بر روی نقشه نمایش داده که شدت آنها و تراکم آنها نیز با توجه به بزرگ و کوچک بودن و رنگ نقاط قابل تشخیص است

برای مثال تصویر 1-3 نقاطی به صورت پراکنده بر روی نقشه داریم که نقطه‌ای از آنها رنگ پررنگ‌تری دارد و به رنگ قرمز متمایل شده است که نشان دهنده این نکته است که تعداد نقاط ثبت شده در این مکان بیشتر است.

نکته دیگری که قابل توجه است با مقایسه دو تصویر به این نکته می‌رسیم که می‌توانیم انتزاع (یا همان abstract) را در این نقش مشاهده نماییم به صورتی که با بزرگ‌نمایی در نقطه‌ای مشخص آن نقطه به نقاطی جدا از یکدیگر تقسیم می‌شود.

از سمت کاربر همانند آنچه که در بالا توضیح داده شد یک دسترسی برای ثبت چاله و دسترسی دیگر برای نمایش نقشه که بر روی آن چاله‌ها بر اساس اهمیت نمایش داده شده‌اند وجود دارد.

نسخه مربوط به کارفرما

کارفرما می‌تواند شهرداری و یا اداره راه و شهرسازی باشد.

کارفرما می‌تواند بر اساس نقشه‌ای که ایجاد شده است تعداد و اهمیت چاله‌های هر نقطه را بررسی نماید و بر اساس آن تصمیمات مورد نیاز را اتخاذ نماید نهاد های دولتی می‌توانند به این نقاط دسترسی داشته باشند و در صورت حل مشکل این نقاط ثبت شده را از روی نقشه حذف کنند.

این برنامه قابلیت گسترش را دارد که می‌تواند نهاد های دیگری همچون گاز ، آب و فاضلاب و برق را نیز شامل شود.

قابلیت‌هایی را که می‌توان اضافه کرد شامل:

- ثبت نشتی لوله گاز و مشکلات گاز

- ثبت مشکلات آب و فاضلاب مانند ترکیدگی لوله

موارد گفته شده تنها بخشی از قابلیت‌هایی است که می‌توانیم به این سیستم اضافه کنیم و آن را به صورت یک برنامه جامع ارائه دهیم و به صورتی باشد که بتوانیم تمام مشکلات شهری و جاده‌ای را در این برنامه بیان نماییم و روند رسیدگی به آنها را نیز زیر نظر داشته باشیم.

منابع و مآخذ

- aprs.gids.nl/nmea
- en.wikipedia.org
- eshop.eca.ir
- shop.aftabrayaneh.com
- openai.com